The Hebrew University of Jerusalem
The Rachel and Selim Benin School of Computer Science and Engineering

The Uniform k-Client Problem

בעיית k־הלקוחות האחידה

Lola Rapoport

A dissertation submitted in partial fulfillment of the requirements
for the Master of Sciences degree
in Computer Science

Under the supervision of **Prof. Yuval Rabani**

September 2023

האוניברסיטה העברית בירושלים
בית הספר להנדסה ולמדעי המחשב על שם רחל וסלים בנין
החוג למדעי המחשב

The Uniform k-Client Problem

בעיית k־הלקוחות האחידה

מוגש על ידי
לולה רפופורט

עבודת גמר לתואר מוסמך במדעי המחשב

עבודה זו הונחתה על ידי
פרופ׳ יובל רבני

ספטמבר 2023

תקציר

רוב המחקר באלגוריתמים מקוונים התמקד במערכות עם תהליכון יחיד, שבהן רק סדרה יחידה של בקשות מתחרה על משאבי המערכת. בעיית κ־הלקוחות, בעיה דואלית לבעיית κ־השרתים הנחקרה היטב, היא בעיה שממדלת מערכת מרובת תהליכונים. בבעיית κ־הלקוחות הבסיסית ניתן מרחב מטרי, יש שרת יחיד ו־κ לקוחות, שכל אחד מהם יוצר סדרה של בקשות שמגיעות באופן מקוון. כל בקשה מאופיינת בנקודה במרחב המטרי. ייתכן שמספר בקשות מאופיינות באותה נקודה. על אלגוריתם מקוון להחליט איזו בקשת לקוח השרת צריך לעבד. על מנת לעבד בקשת לקוח, האלגוריתם המקוון מזיז את השרת במרחב המטרי לנקודה המתאימה לבקשה. בהינתן פונקציית עלות, המטרה של הבעיה היא למזער את העלות של עיבוד כל κ הסדרות של הלקוחות.

אנו מנתחים את הגרסה האחידה של בעיית κ־הלקוחות, שבה המרחב המטרי הניתן הוא מרחב מטרי אחיד. אנו מחשבים את ביצועי האלגוריתמים באמצעות שתי פונקציות עלות: מרחק כולל (בעיקר) וזמן השלמה ממוצע. בנוסף, אנו מגדירים משפחה של אלגוריתמים מקוונים, שבה כל אלגוריתם נקרא עצלן. שלוש מהתוצאות העיקריות שאנו משיגים הינן:

- חסם עליון כללי של k על יחס התחרותיות של אלגוריתמים עצלנים עבור שתי פונקציות העלות.

- חסמים הדוקים על הביצוע של מספר אלגוריתמים סטנדרטיים, לפי פונקציית עלות המרחק הכולל.
- חסם תחתון דטרמיניסטי של $\frac{\lfloor \log k \rfloor}{2} + 1$ וחסם תחתון רנדומי של $\Omega(\log \log k)$ על יחס התחרות של כל אלגוריתם מקוון עבור פונקציית עלות המרחק הכולל.

אנו גם חוקרים וראיציות חדשות של בעיית k-הלקוחות עם פונקציית עלות המרחק הכולל, כאשר יש מונית משרתת וכל בקשה מזוהה עם זוג נקודות במרחב המטרי (נקודת התחלה ונקודת יעד), וכאשר יש באפר מסדר עם קיבולת שניתן להשתמש בו כדי לאחסן בו בקשות מכל הלקוחות.

Abstract

Most of the research in online algorithms has focused on *single-threaded systems*, where only a *single* sequence of requests compete for system resources. The k-client problem, a dual problem of the well-studied k-server problem, is a problem that models *multi-threaded online systems*. In the basic k-client problem, a metric space is given, there is a single server and k clients, each of which generates a sequence of requests arrive online. Each request is characterized by a point in the metric space. It is possible that multiple requests are characterized by the same point. An online algorithm must decide which client's request the server should process. To process a client's request, the online algorithm moves the server in the metric space to the point associated with that request. Given a cost function, the goal of the problem is to minimize the cost of processing all k client sequences.

We analyse the *uniform k-client problem*, a variant of the k-client problem in which the given metric space is an uniform metric space. We evaluate the performance of algorithms using two cost functions: total distance (mostly) and average completion time. In addition, we define a family of online algorithms, in which any algorithm is called *lazy*. Three of main

results we obtain are:

- A general upper bound of k on the competitive ratio of lazy algorithms for both cost functions.
- Tight bounds on the performance of several standard algorithms using the total distance cost function.
- A deterministic lower bound of $\frac{\lfloor \log k \rfloor}{2} + 1$ and a randomized lower bound of $\Omega(\log \log k)$ on the competitive ratio of any online algorithm for the total distance cost function.

We also consider new variations of the k-client problem with the total distance cost function, where there is a serving taxi and each request is associated with a pair of points in the metric space (start point and destination point), and where there is a reordering buffer with capacity which can be used to store in it requests from all clients.

Acknowledgements

First and foremost, I am very grateful to my supervisor, Prof. Yuval Rabani, for guiding and advising me throughout this research. The research process has been incredible due to his great devotion and range of interests.

Also, I want to give thanks to the employees of the Rachel and Selim Benin School of Computer Science and Engineering at the Hebrew University of Jerusalem for their assistance during the last two years.

Finally, I would like to thank my mother, Victoria, for her endless support and unconditional love.

Contents

List of Figures

List of Algorithms

Chapter 1

Introduction

1.1 Online Computation

Over the past 40 years there has been a great growth in studies of online optimization problems. Usually, when we design algorithms for solving optimization problems, we assume that the entire input is known is advance. However, this may not be the case as in the Ski Rental Problem, List Processing, Paging and many other problems presented in the literature and highly motivated by many real-life scenarios. We restrict our attention to only online minimization problems, and it is important to note everything below only applies to such problems.

Online algorithms attempt to model a real-life situation, where the complete data is unknown at the beginning of their execution. In such cases, the input is given incrementally, and the online algorithm has to make decisions in real time to react to the input, considering that it will not be able to change the past actions later. Therefore, since decisions are made irreversibly, the best action taken at the current step may prove to be not very good later. The sequence of actions taken has an associated cost w.r.t. a given *cost function*.

In general, the online algorithm is compared to an optimal offline algorithm that knows the entire input in advance. For minimization problems, the optimal offline problem algorithm is defined for each entire input, as an algorithm that minimizes the cost of processing that input w.r.t. a given cost function. Intuitively, since online algorithms lack future information, it is clear that they may perform much worse than optimal offline algorithms.

However, it is not simple to define a measure of the quality of the performance for online algorithms, since no matter what decisions an online algorithm made in response to an initial set of requests, there may be probably a sequence of further requests that makes the first decisions to be counterproductive w.r.t. to a given objective function. There are several ways to measure the quality of

an online algorithm. The widely accepted among them is *competitive analysis*. This field formally introduced by Sleator and Tarjan [20] and since then it has been steadily developed. As explained above, competitive analysis addresses the comparison between the performance of the online algorithm and the optimal offline algorithm. The *competitiveness* of an online algorithm is the ratio of its performance to the performance of an optimal offline algorithm. There are several (non-equivalent) definitions for competitiveness in minimization problems, and only one of them is used in this work. We now define competitiveness of deterministic algorithms. While the deterministic case is straightforward, the randomized case is much more subtle and is addressed in the next section. To define competitiveness of deterministic algorithms, let $\mathcal{A}$ be an online deterministic algorithm for some minimization problem. We denote by $C^{\mathcal{A}}(I)$ and by $C^{\mathcal{OPT}}(I)$ the cost (w.r.t. some cost function) incurred by an algorithm $\mathcal{A}$ and an optimal offline algorithm in satisfying an input I, respectively.

Definition 1.1.

- $\mathcal{A}$ is called α-*competitive* ($\alpha \geq 1$) if

  $$C^{\mathcal{A}}(I) \leq \alpha \cdot C^{\mathcal{OPT}}(I) + \tau,$$

  for each input I, where $\tau \geq 0$ is a constant independent of the input.

- The *competitive ratio* of $\mathcal{A}$ is defined to be the infimum over all α such that $\mathcal{A}$ is α-competitive.

- $\mathcal{A}$ is called strictly α-competitive ($\alpha \geq 1$) if

  $$C^{\mathcal{A}}(I) \leq \alpha \cdot C^{\mathcal{OPT}}(I),$$

  for each input I.

- The *strict competitive ratio* of $\mathcal{A}$ is defined to be the infimum over all α such that $\mathcal{A}$ is strictly α-competitive. Alternatively, one can define the strict competitive ratio of $\mathcal{A}$ as

  $$\sup_I \frac{C^{\mathcal{A}}(I)}{C^{\mathcal{OPT}}(I)}. \tag{1.1.1}$$

Remark 1.2. The definition of α-competitiveness varies in the literature. Often an online algorithm is called α-competitive by the definition above. Yet, some authors prefer to use the strict definition of α-competitiveness (e.g., [1]). Namely, they define an online α-competitive algorithm by only the above definition for strictly α-competitive algorithms. In this work, we will stick to the definition given above.

It is a usual practice to refer online algorithms as a game between an online player and an *adversary*. Given the online algorithm, this adversary selects an input for the problem so as to make the online algorithm perform as bad as

possible. Formally speaking, the adversary tries to maximize the ratio between the costs of the online algorithm and optimal offline algorithm.

Presenting the problem in this manner enables us to obtain both lower and upper bounds. Each lower bound l indicates that no online algorithm (for the same minimization problem) has a lower competitive ratio than l, that is, no online algorithm is better than l-competitive. The result in Lemma 1.9 is useful to obtain lower bounds for the competitiveness of online algorithms. On the other hand, each upper bound u is generally achieved by providing an online algorithm that is u-competitive.

We refer to [21] for more regarding the literature on online optimization.

1.1.1 Adversaries Versus Randomization

Randomization has gained more focus considering that, in order to construct inputs, adversaries predict the deterministic online algorithms' behavior and try to force them perform as bad as possible with respect to their offline optimal strategies. The goal of randomization is to weaken the adversary's ability to predict.

The main feature of randomized algorithms is that they toss coins during their execution. This feature complicates the adversary's objective, who may try to thwart the algorithm along a path that the randomized online algorithm will not necessarily follow.

In contrast with the deterministic case, for the randomized case different types of adversaries have been defined. For all of them, it is assumed that the adversary knows the code of the randomized online algorithms, but does not know the results of the random coin tosses. We present the following three types of adversaries:

- $\mathcal{OBL}$ (oblivious): This adversary is called *oblivious* reflecting the fact that he is oblivious to the random choices made by the online algorithm. Thus, the adversary generates an input for the problem without knowing any result of the coin tosses made by the randomized online algorithm and pays optimally.
- $\mathcal{ADON}$ (adaptive-online): The adversary chooses each request of the input after having observed the *previous* choices of the online algorithm. In addition, this adversary decides what action to follow without knowing the future requests.
- $\mathcal{ADOF}$ (adaptive-offline): The adversary chooses each request of the input after having observed the *previous* choices of the online algorithm. In contrast with the adaptive-online adversary, the adaptive-offline adversary

decides what action to take at each step knowing the whole input and the corresponding online algorithm behavior.

Summing up, the advantage of the *adaptive* over the *oblivious* adversary is that the first decides what request to generate after knowing the past behavior of the online algorithm. Furthermore, the advantage of the *adaptive-offline* adversary over the *adaptive-online* adversary is that the first is allowed to serve its input in an offline fashion at an optimal cost (i.e., it can wait with processing the input until it is finished), while the second has to serve its input in an online fashion.

We are now ready to define competitiveness of randomized algorithms using the 3 types of adversaries defined above.

Definition 1.3. Let $\mathcal{A}$ be an online randomized algorithm for some minimization problem.

- $\mathcal{A}$ is called α-competitive ($\alpha \geq 1$) against any *oblivious* adversary if is a constant $\tau \geq 0$ such for all inputs I generated by an oblivious adversary,

$$\mathbb{E}\left[C^{\mathcal{A}}(I)\right] \leq \alpha \cdot C^{\mathcal{OPT}}(I) + \tau,$$

  where the expectation is taken over the random choices made by $\mathcal{A}$.

- $\mathcal{A}$ is called α-competitive ($\alpha \geq 1$) against any *adaptive-online* adversary if there is a constant $\tau \geq 0$ such for all adaptive-online adversaries ADV,

$$\mathbb{E}\left[C^{\mathcal{A}}(I)\right] \leq \alpha \cdot \mathbb{E}\left[C^{\mathsf{ADV}}(I)\right] + \tau,$$

  where the expectation is taken over the random choices made by $\mathcal{A}$ and I denotes the generated input by ADV.

- $\mathcal{A}$ is called α-competitive ($\alpha \geq 1$) against any *adaptive-offline* adversary if there is a constant $\tau \geq 0$ such for all adaptive-offline adversaries ADV,

$$\mathbb{E}\left[C^{\mathcal{A}}(I)\right] \leq \alpha \cdot \mathbb{E}\left[C^{\mathcal{OPT}}(I)\right] + \tau,$$

  where the expectation is taken over the random choices made by $\mathcal{A}$ and I denotes the generated input by ADV.

- **Equivalent General Definition.** $\mathcal{A}$ is called α-competitive ($\alpha \geq 1$) against any adversary of type $C \in \{\mathcal{OBL}, \mathcal{ADON}, \mathcal{ADOF}\}$ if there is a constant $\tau \geq 0$ such for all adversaries of type C, ADV,

$$\mathbb{E}\left[C^{\mathcal{A}}(I)\right] \leq \alpha \cdot \mathbb{E}\left[C^{\mathsf{ADV}}(I)\right] + \tau,$$

  where the expectation is taken over the random choices made by $\mathcal{A}$ and I denotes the generated input by ADV.

Given any online problem, one can show the following theorem.

Theorem 1.4. *For all randomized online algorithms $\mathcal{A}$, if we let $R^{\mathcal{OBL}}$, $R^{\mathcal{ADON}}$ and $R^{\mathcal{ADOF}}$ denote the competitive ratios considering oblivious adversaries, adaptive-online adversaries and adaptive-offline adversaries, respectively. We have that*

$$R^{\mathcal{OBL}} \leq R^{\mathcal{ADON}} \leq R^{\mathcal{ADOF}}.$$

According to the above theorem, any lower bound for an online randomized algorithm against oblivious adversaries is also a lower bound for it against the other two types of adversaries. Therefore, from competing against oblivious adversaries one can obtain lower bounds for an online randomized algorithm against all 3 types of adversaries.

Ben-David et al. [22] investigated the strength of the adversaries w.r.t. an arbitrary online problem with a sequence of requests as input and showed the following statements.

Theorem 1.5. *Let $\mathcal{A}$ be a randomized online algorithm that is α-competitive against any adaptive-offline adversary, then there also exists an α-competitive deterministic online algorithm.*

The above theorem indicates that randomization does not help against adaptive-offline adversaries, so we will ignore them when looking for improved competitive ratios.

Theorem 1.6. *Let $\mathcal{A}$ be a randomized online algorithm that is α-competitive against any adaptive-online adversary. If there is also a β-competitive algorithm against any oblivious adversary, then $\mathcal{A}$ is $(\alpha \cdot \beta)$-competitive against any adaptive-offline adversary.*

The following corollary is an immediate consequence of the above two theorems.

Corollary 1.7. *Let $\mathcal{A}$ be a randomized online algorithm. If $\mathcal{A}$ is α-competitive against any adaptive-online adversary, then there is an α^2-competitive deterministic online algorithm.*

1.1.1.1 Yao's Minimax Principle

Given the sequence of random bits used, a randomized algorithm acts deterministically. Therefore, one may consider a randomized algorithm as a random choice from a distribution of deterministic algorithms. Let X be the space of problem inputs and A be the space of all possible deterministic algorithms. Denote probability distributions over X and A by $p_x = \Pr[X = x]$ and $q_a = \Pr[A = a]$, respectively. Define $c(a, x)$ as the cost of algorithm $a \in A$ on input $x \in X$.

Theorem 1.8 ([3])**.**

$$C = \max_{x \in X} \mathbb{E}_q [c(A, x)] \geq \min_{a \in A} \mathbb{E}_p [c(a, X)] = D$$

Proof.

$$
\begin{aligned}
C &= \sum_{x \in X} p_x \cdot C && \text{Sum over all possible inputs} \\
&\geq \sum_{x \in X} p_x \mathbb{E}_q \left[c(A, x)\right] && \text{Since } C = \max_{x \in X} \mathbb{E}_q \left[c(A, x)\right] \\
&= \sum_{x \in X} p_x \sum_{a \in A} q_a \cdot c(a, x) && \text{By definition of } \mathbb{E}_q \left[c(A, x)\right] \\
&= \sum_{a \in A} q_a \sum_{x \in X} p_x \cdot c(a, x) && \text{Swap summations} \\
&= \sum_{a \in A} q_a \mathbb{E}_p \left[c(a, X)\right] && \text{By definition of } \mathbb{E}_p \left[c(a, X)\right] \\
&\geq \sum_{a \in A} q_a \cdot D && \text{Since } D = \min_{a \in A} \mathbb{E}_p \left[c(a, X)\right] \\
&= D && \text{Sum over all possible algorithms}
\end{aligned}
$$

■

Implication. If one can determinate that *any* deterministic algorithm cannot do well on a given distribution of random inputs, then no randomized algorithm can do well on *all* inputs. In other words, one can derive a lower bound on the expected performance of any randomized algorithm on a worst-case input instance by deriving a lower bound on the expected performance of any deterministic algorithm on a particular distribution of random inputs.

1.1.2 Deriving Lower Bounds

The following result is suitable for any online problem and is useful to obtain lower bounds for the competitiveness of online algorithms.

Lemma 1.9. *Given an online deterministic algorithm $\mathcal{A}$, if there exists a constant $\beta \geq 0$ such that for every integer n, there exists some input instance I^n such that:*

1. $C^{\mathcal{OPT}}(I^n) \geq n$ *and*
2. $C^{\mathcal{A}}(I^n) \geq \alpha \cdot C^{\mathcal{OPT}}(I^n) - \beta$,

then $\mathcal{A}$ is not better than α-competitive.

Proof. Suppose to the contrary that $\mathcal{A}$ is $(\alpha - \varepsilon)$-competitive for some $\varepsilon > 0$. Then there exists some constant τ such that for all I,

$$C^{\mathcal{A}}(I) \leq (\alpha - \varepsilon) \cdot C^{\mathcal{OPT}}(I) + \tau. \tag{1.1.2}$$

Let n be some integer such that $n > \frac{\beta + \tau}{\varepsilon}$. Let I^n be an input instance that satisfies condition (1) and condition (2) above for n. Since I^n satisfies condition

(2) above for n and from (1.1.2) for I^n, we have

$$\begin{aligned}\tau \geq C^{\mathcal{A}}(I^n) - (\alpha - \varepsilon) \cdot C^{\mathcal{OPT}}(I^n) &\geq \alpha \cdot C^{\mathcal{OPT}}(I^n) - \beta - (\alpha - \varepsilon) \cdot C^{\mathcal{OPT}}(I^n) \\ &= \varepsilon \cdot C^{\mathcal{OPT}}(I^n) - \beta.\end{aligned}$$

Therefore, we have $C^{\mathcal{OPT}}(I^n) \leq \frac{\beta+\tau}{\varepsilon}$, a contradiction to condition (1) which states that $C^{\mathcal{OPT}}(I^n) > \frac{\beta+\tau}{\varepsilon}$ since $n > \frac{\beta+\tau}{\varepsilon}$. Therefore, $\mathcal{A}$ is no better than α-competitive. ■

Note 1.10. We proved the result above only for online deterministic algorithms. One can extend the result for online randomized algorithms against adversaries of type $C \in \{\mathcal{OBL}, \mathcal{ADON}, \mathcal{ADOF}\}$ using a proof similar to the proof for the deterministic case and obtain the following lemma.

Lemma 1.11. *Let $C \in \{\mathcal{OBL}, \mathcal{ADON}, \mathcal{ADOF}\}$. Given an online randomized algorithm $\mathcal{A}$, if there exists a constant $\beta \geq 0$ such that for any integer n, there exists an adversary ADV^n of type C such that:*

1. $\mathbb{E}\left[C^{\mathsf{ADV}^n}(I)\right] \geq n$ *and*
2. $\mathbb{E}\left[C^{\mathcal{A}}(I)\right] \geq \alpha \cdot \mathbb{E}\left[C^{\mathsf{ADV}^n}(I)\right] - \beta$[1]*,*

then $\mathcal{A}$ is not better than α-competitive against adversaries of type C.

1.2 The k-Client Problem

1.2.1 Motivation and Problem Statement

Most of the research in online algorithms has been dedicated to single-threaded systems, where only a single sequence of requests compete for system resources. In particular, at any given time, there is at most one outstanding request in the system and future requests will not arrive until the current request is serviced[2]. Hence, the underlying problem addressed by most of the previous work in online algorithms has been deciding which system resource(s) should be used in order to service the current request.

While the single request sequence model captures many important problems (e.g., the paging problem or the k-server problem), there are problems which do not belong to this group. In a typical such problem, there is a single system resource such as processor, at any given time, there are multiple requests in the system. Consequently, the underlying problem is deciding which current request should the system service rather than which system resource to use. Notice that there are cases where there are multiple resources and multiple requests, so the

[1] In both conditions, the expectation is taken over the random choices made by $\mathcal{A}$ and I denotes the generated input by ADV^n.

[2] There are problems where the system is allowed to choose to not service the current request.

system has to decide, at any given time, which request to service as well as which system resource to use.

The problem with this multiple request model is that it loses the thread-based nature of the single request model, in which the arrival of requests is dependent on whether or not the system processed previous requests. The thread-based client model depicts some practical applications which cannot be depicted by the multiple request model. For example, the problem on line metrics models the disk scheduling problem in a multi-threaded environment, where for relatively long stretches of time there is a fixed number of users on an operating system making requests to the disk as follows. Each user generates a sequence of requests for service where each request is only generated after the previous request of that client has been serviced. In addition, the thread-based client model also describes database systems where users perform transactions which constitute a series of atomic operations in the database.

In order to capture the thread-based client model, Alborzi et al. [1] defined the k-client problem. The k-client problem, its variations and some online problems we mention involve a metric space, which defined as follows.

Definition 1.12 (Metric Space)**.** A pair $\mathcal{M} = (M, d)$ where M is a set of points and $d : M \times M \to \mathbb{R}^+$ is a distance function, is a *metric space*, if for all $p, q, r \in M$:

1. $d(p, q) = 0 \Leftrightarrow p = q$ (Reflexivity)
2. $d(p, q) = d(q, p)$ (Symmetry)
3. $d(p, r) \leq d(p, q) + d(q, r)$ (Triangle inequality)

The k-client problem consists of serving k client sequences using one server over some metric space $\mathcal{M} = (M, d)$. More specifically, each client generates a sequence of requests arrive online. Each request is characterized by a point in the metric space. It is possible that multiple requests are characterized by the same point. An online algorithm must decide, at any given time, which client's request the server should service. To service a client's request, the online algorithm moves the server in the metric space at constant speed to the point associated with that request. So, we assume zero processing time for all requests.

We use the following notation to define the input instance for the k-client problem. The input instance for the problem is denoted by I and is a tuple of k client sequences. That is, $I = \langle I_1, \ldots, I_k \rangle$, where $I_i \subseteq I$ denotes the sequence of requests generated by client i, for $1 \leq i \leq k$. We denote the number of requests generated by client i by n_i, for $1 \leq i \leq k$. For simplicity, we assume that each client has a dummy request located at the initial position of the server. Also, we denote by $r_{i,j}$ the j-th request of client i for $1 \leq i \leq k$ and $0 \leq j \leq n_i$. Notice that $r_{i,0}$ represents the dummy request of client i, for $1 \leq i \leq k$.

At any time, each client has at most one request in the system. That is to

say, request $r_{i,j+1}$ arrives exactly when $r_{i,j}$ has been serviced for $1 \leq i \leq k$ and $0 \leq j \leq n_i - 1$. Therefore, for $1 \leq i \leq k$, $r_{i,0}$, the dummy request of client i is serviced at time 0 and $r_{i,1}$ is the first real request of client i that arrives at time 0. In [1], 3 cost functions were considered for the basic k-client problem:

1. The total distance cost function (also known as *makespan*), which measures the total distance moved by the server[3]. This is the cost function used in the reordering buffer management problem and the k-server problem. Notice that assuming the server never idles unnecessary, this cost function is equivalent to the maximum completion time cost function. The maximum completion time cost function measures the maximum completion time of any *entire* client; that is, we take the maximum of the completion times of requests r_{i,n_i}, for $1 \leq i \leq k$.

2. The average completion time cost function, which measures the average completion time of any *entire* client; that is, we take the average of the completion times of requests r_{i,n_i}, for $1 \leq i \leq k$.

3. The maximum response time cost function, which measures the maximum response time of any *individual request* of any client; that is, the maximum time between the servicing of two consecutive requests of any client.

We do not consider the maximum response time cost function in this work.

1.2.1.1 Lazy Algorithms on Uniform Metrics

The k-client problem on an uniform metric space is called the *uniform k-client problem.* When considering the k-client problem on an uniform metric space (M, d), each request is associated with a point in the metric space and for each two spaced points p and q the distance between them is 1, i.e., $d(p, q) = 1$. For the special case of the k-client problem on an uniform metric space we can consider, without loss of generality, only *lazy* algorithms defined as follows.

Definition 1.13 (Lazy Algorithm)**.** An algorithm for the uniform k-client problem is called *lazy* if it fulfills the following two properties:

- It only moves the server towards a request that is not at the current position.
- It serves the requests at the current position as long as there are requests.

Note that every (in particular every optimal offline) algorithm can be converted to a lazy algorithm without increasing the cost.

[3] In a generalized version of the problem where there are multiple serves, this cost function measures the total distance moved by *all* the servers.

1.2.2 Related Work

As mentioned earlier, the k-client problem was defined by Alborzi et al. [1], who studied the problem for 3 cost functions: total distance, average completion time and maximum response time. They gave (strictly) $(2k-1)$-competitive algorithms for the k-client problem for both the total distance and average completion time cost functions, which are suitable on any metric space. They proved that these algorithms are not better than $(2k-1)$-competitive on any metric space containing the continuous line. For the problem with n uniformly-spaced points on a line, they showed a competitive algorithm with a competitive ratio that depends only on n (that is, independent of k), for both the total distance and average completion time cost functions. For the total distance cost function, they proved a deterministic lower bound of $\Omega(\log k)$ (more specifically, $\frac{\log k}{2}+1$ when k is a power of 2 and $\frac{\log k}{2}$ when k is not a power of 2) and a randomized lower bound of $\Omega(\log\log k)$ against oblivious adversaries for the problem on an uniform metric space, only according to the strict definition of a competitive ratio. They also proved the same deterministic lower bound for the problem on line metrics, according to the usual definition of competitive ratio. For the average completion time cost function, they proved a deterministic lower bound of $\Omega(\log k)$ (more specifically, $\frac{\log k}{2}+1$ when k is a power of 2 and $\frac{\log k}{2}$ when k is not a power of 2) for the problem on uniform and line metrics, according to the usual definition of competitive ratio. For the very restricted case $k=2$, they showed a deterministic lower bound of 1.8 for the problem on an uniform metric space for both the total distance and average completion time cost functions. Furthermore, when $k=2$, they proved a deterministic lower bound of $\frac{25}{9}$ and 3 for the problem on the continuous line for the total distance and average completion time cost functions, respectively.

Two generalized versions of the k-client problem were studied in [1]. In one version, requests may have non-zero processing times and instead of the total distance, the maximum completion time is used as a cost functions (this is distinct from the total distance cost function and more relevant, in case there are processing times). For the maximum completion and average completion time cost functions, they showed that the worst case of most of their considered algorithms occurs when requests have zero processing times. Essentially, they showed that processing times do not make the problem more complex. The other version is called the *k-client l-server problem* and has multiple clients and multiple servers. They showed a competitive deterministic online algorithm for this version by employing a competitive deterministic online algorithm for the l-server problem for each client sequence one by one.

Feuerstein and Strejilevich de Loma [2] investigated the multi-threaded paging (MTP) problem only for the total distance cost function. This is a special case of the k-client l-server problem where the underlying metric space is uniform and a generalized version of the uniform k-client problem where there multiple servers. They designed a lk-competitive deterministic online algorithm by em-

ploying the best possible deterministic online algorithm for each client sequence sequentially. A deterministic lower bound of $l + 1 - \frac{1}{k}$ was also derived. In particular, a deterministic lower bound of $2 - \frac{1}{k}$ was obtained for the uniform k-client problem with the total distance cost function. While Feuerstein and Strejilevich de Loma investigated deterministic algorithms for the MTP problem, Seiden [8] studied randomized algorithms for this problem. The offline version of MTP has been studied as well for the total distance and average completion time cost functions. Rehn-Sonigo et al. [24] proved for both these cost functions that MTP with a fixed number of servers is NP-hard and that there are no constant-approximation algorithms unless $P = NP$, even in the simplest case where there is a single server. Hence, in particular they proved for both these cost functions that the uniform k-client problem is NP-hard and that there are no constant-approximation algorithms assuming that $P \neq NP$.

1.3 Reordering Buffer Management

1.3.1 Problem Statement

One of the variations of the k-client problem we explore involves the reordering buffer management problem. In the reordering buffer management problem (RBM), given a metric space $\mathcal{M} = (M, d)$, the input is an online sequence $I = r_1 r_2 \dots r_N$ of N requests, where each request corresponds to a point in M. There is a server that moves from point to point to process these requests. In addition, a *reordering buffer* which is a random access buffer with a storage for l requests that can be used by an online algorithm in order to rearrange the input sequence. Each input request r_i that is not handled yet, can be stored in the buffer, as long as the buffer is not full. Once the buffer is full, at least one of the requests currently stored in the buffer must be removed and processed by the algorithm. To process a request stored in the buffer, the algorithm moves the server in the metric space to the point corresponding to that request. The removed requests create an *output sequence*, which is a permutation of I. We suppose that the reordering buffer is initially empty, and after processing the entire input sequence, the buffer is empty again. The goal is to minimize the total distance that the server has to travel in order to process the entire input sequence, which is the cost of processing the sequence.

1.3.2 Related Work

Most of the work for RBM has been dedicated to online algorithms; their performance is analyzed by the *competitive analysis*. The problem, on an uniform metric space, was introduced by Räcke et al. [4], who studied several standard strategies and proved that the competitive ratio of the First In First Out (FIFO) and the Least Recently Used (LRU) strategies is $\Omega\left(\sqrt{l}\right)$ and the competitive ratio of the Most Common First strategy is $\Omega(l)$. Moreover, a deterministic $O\left(\log^2 l\right)$-competitive algorithm is provided in [4]. Afterwards, Englert and

Westermann [10] designed a deterministic $O(\log l)$-competitive algorithm, and then Avigdor-Elgrabli and Rabani [11] improved this result by presenting a deterministic $O\left(\frac{\log l}{\log\log l}\right)$-competitive algorithm. The current best known deterministic upper and lower bounds for the competitive ratio are $O\left(\sqrt{\log l}\right)$ and $\Omega\left(\sqrt{\frac{\log l}{\log\log l}}\right)$, respectively, proved by Adamaszek et al. [12]. In addition, a randomized lower bound for the competitive ratio of $\Omega(\log\log l)$ is proved in [12]. Later, Avigdor-Elgrabli and Rabani [13] proved that this lower bound for randomized algorithm is tight by showing a randomized algorithm with a matching upper bound.

The offline version of the problem on an uniform metric space has been studied as well. It was proven independently to be NP-hard by both Chan et al. [6] and Asahiro et al. [7]. Avigdor-Elgrabli and Rabani [14] provided the first polynomial time constant factor approximation algorithm for the offline version of RBM on an uniform metric space.

However, for other metric spaces, the picture is less clear. Khandekar and Pandit [16] investigated the problem on the line metric, which is motivated by its application to the disk scheduling problem. They presented a randomized algorithm that is $O\left(\log^2 n\right)$-competitive against an oblivious adversary for n uniformly-spaced points on a line, and then Gamzu and Segev [5] improved this by providing a deterministic $O(\log n)$-competitive algorithm. Later, Englert [9] gave an alternative deterministic $O(\log n)$-competitive algorithm. This algorithm avoids more complicated moves of the server compered to the algorithm in [5] and the upper bound on the competitive ratio is slightly improved. Furthermore, Gamzu and Segev [5] a deterministic $O(\log N \log\log N)$-competitive algorithm for the continuous line, where N is the number of the requests in the input. Also, they proved a deterministic lower bound of $1+\frac{2}{\sqrt{3}} \approx 2.154$ on the competitive ratio for the line metric. This is the only known lower bound for the problem on the line.

As for more general class of metric spaces, Englert and Westermann [15] showed that a reordering buffer of size l can only reduce the cost of the input by $2l-1$ at most, on any metric space. Thus, they proved a basic "do nothing" upper bound of $O(l)$ on the competitive ratio for all metric spaces. For general metric spaces with n points, Englert et al. [17] showed a randomized $O\left(\log^2 l \cdot \log n\right)$-competitive algorithm. This algorithm is based on a deterministic $O\left(\log^2 l\right)$-competitive online algorithm for hierarchically separated tree (HST) metrics presented in [17]. Both the $O(\log n)$ factor and the randomness derive from using an approximating of arbitrary metrics by distributions on HST metrics, given by Fakcharoenphol et al. [19]. Later, Englert and Räcke [18] improved this result on the expected competitiveness for general finite metric spaces to $O(\log l \cdot \log n)$. Kohler and Räcke [23] gave a deterministic $O(\log\Delta + \min\{\log l, \log n\})$-competitive online algorithm for general metric

spaces with n points and aspect ratio Δ.

1.4 Results and Outline

In this work, we discuss the uniform k-client problem and expand the model of the k-client problem to include a severing taxi and a reordering buffer. In Chapter 2, we focus on the uniform k-client problem. In particular, we prove an upper bound of k on the strict competitive ratio of any lazy algorithm for both the total distance and average completion time cost functions. Furthermore, we show several standard algorithm that are exactly k-competitive for the total distance cost function. We also prove a deterministic lower bound of $\frac{\lfloor \log k \rfloor}{2} + 1$ and a randomized lower bound of $\Omega(\log \log k)$ for the total distance cost function. In addition, we prove a constant randomized lower bound of 1.25 when $k \geq 2$ for the total distance cost function, which may be handy to the problem when $k = 2$.

Then, in Chapter 3, we consider the k-client 1-taxi problem, a generalized version of the k-client problem with the total distance cost function, where there is a serving taxi and each request is associated with a pair of points in the metric space (start point and destination point). We prove that the difference between the best possible deterministic and/or randomized (against oblivious adversaries) competitive ratios of the k-client 1-taxi problem and the basic k-client problem is at most 2.

Finally, in Chapter 4, we consider the k-client l-buffer problem, a generalized version of the k-client problem with the total distance cost function, where there is a reordering buffer with capacity which can be used to store in it requests from all clients. We prove a general result which shows that any deterministic competitive algorithm for the reordering buffer management problem can be used to produce a deterministic competitive algorithm for the k-client l-buffer problem.

In Chapter 5, we summarize our work and offer some open problems and possible research directions.

Chapter 2

The Basic Uniform k-Client Problem

Recall that Alborzi et al. [1] gave strictly $(2k-1)$-competitive algorithms for the k-client problem for both the total distance and average completion time cost functions, which are suitable on any metric space. Also, they proved a deterministic lower bound of $\Omega(\log k)$ and a randomized lower bound of $\Omega(\log \log k)$ against oblivious adversaries for the problem on an uniform metric space, only according to the strict definition of a competitive ratio.

In this chapter, we consider the basic k-client problem where the underlying metric space is an uniform metric space. We consider the problem for the total distance (mostly) and average completion time cost functions. Throughout this chapter, we denote an uniform metric space with n points by $\mathcal{U}_n$ and an uniform metric space with a countably infinite points by $\mathcal{U}_\infty$. The main results of this chapter (in order) are as follows.

We start by showing tight bounds on the performance of several standard algorithms using the total distance cost function. Then, we prove a deterministic lower bound of $\frac{\lfloor \log k \rfloor}{2} + 1 = \Omega(\log k)$ and a randomized lower bound of $\Omega(\log \log k)$ for the total distance cost function. Both these lower bounds are obtained by generalizing the constructions of the lower bounds given in [1]. We significantly improve the previously known deterministic lower bound of $2 - \frac{1}{k}$ proved in [2]. Indeed, the previous lower bound is upper-bounded by 2 regardless of the value of k, while the lower bound we obtain grows logarithmically as a function of k. Finally, we prove an upper bound of k on the strict competitive ratio of any lazy algorithm for both the total distance and average completion time cost functions. Consequentially, for the uniform k-client problem with the strict definition of the competitive ratio, we slightly improve the upper bound (from$(2k-1)$ to k) for both the total distance and average completion time cost function.

2.1 Basic Algorithms and Their Competitive Ratio

2.1.1 The Total Distance Cost Function

Throughout this section, only the total distance cost function is considered. We first consider the Sequential ($\mathcal{SEQ}$) algorithm, which arbitrary selects an order of the clients from 1 to k. After that, it services all the requests of the first client, then all the requests of the second client, and so on. Notice that if the server passes over requests from other clients while serving the current client, it will still serve them. In addition, the server service requests corresponding to a point, as long as there are requests at that point. The $\mathcal{SEQ}$ algorithm is described in Algorithm 2.1.

Algorithm 2.1 The $\mathcal{SEQ}$ Algorithm

1: **Require:** initial position of the server p_0
   k clients who each have a sequence of requests starting at p_0
2: select an arbitrary order of the clients from 1 to k.
3: **for** $i \leftarrow 1$ to k **do**
4: serve all the requests of the i-th client.
5: **end for**

In [1], it was proven that $\mathcal{SEQ}$ is strictly $(2k-1)$-competitive for the k-client problem on any metric space. We now show that this upper bound drops to k when considering the uniform varient of the problem, i.e. $\mathcal{SEQ}$ is strictly k-competitive for the k-client problem on any uniform metric space.

Theorem 2.1 (Upper Bound for $\mathcal{SEQ}$: Total Distance)**.** *The $\mathcal{SEQ}$ algorithm is strictly k-competitive for the k-client problem on any uniform metric space.*

Proof. Given an input instance I that consists of requests in some uniform metric space, let the clients be ordered according to the order chosen by the $\mathcal{SEQ}$ algorithm. Recall that for $1 \leq i \leq k$, $I_i \subseteq I$ denotes the set of requests of client i (the dummy requests at the initial position of the server + n_i requests). Let $\widetilde{n}_i$ denote the number of sequences of identical non-dummy requests in I_i, for $1 \leq i \leq k$. Note that for any $1 \leq i \leq k$, $\widetilde{n}_i$ is exactly the optimal cost of serving the set of requests of client i, i.e. $\widetilde{n}_i = C^{\mathcal{OPT}}(I_i)$, since all the requests are in an uniform metric space. The cost of serving client 1 is exactly $\widetilde{n}_1$ and the cost of serving client i is upper-bounded by $d(r_{i-1,n_{i-1}}, r_{i,1}) + (\widetilde{n}_i - 1)$ for $2 \leq i \leq k$ (the cost of moving to serve client i + the maximal cost of serving requests of client i). Since

$$d(r_{i-1,n_{i-1}}, r_{i,1}) = \begin{cases} 1, & \text{if } r_{i-1,n_{i-1}} \neq r_{i,1} \\ 0, & \text{otherwise} \end{cases}$$

for $2 \leq i \leq k$ (as we consider an uniform metric space). we have that

$d(r_{i-1,n_{i-1}}, r_{i,1}) \leq 1$. Thus,

$$C^{\mathcal{SEQ}}(I) \leq \widetilde{n}_1 + \sum_{i=2}^{k} \big(d(r_{i-1,n_{i-1}}, r_{i,1}) + (\widetilde{n}_i - 1)\big) \leq \widetilde{n}_1 + \sum_{i=2}^{k} 1 + (\widetilde{n}_i - 1)$$
$$= \sum_{i=1}^{k} \widetilde{n}_i. \quad (2.1.1)$$

Let $\mathcal{OPT}$ be an optimal offline algorithm. We have

$$\sum_{i=1}^{k} \widetilde{n}_i = \sum_{i=1}^{k} C^{\mathcal{OPT}}(I_i) \leq \sum_{i=1}^{k} C^{\mathcal{OPT}}(I) = k \cdot C^{\mathcal{OPT}}(I). \quad (2.1.2)$$

The above inequality holds since $\widetilde{n}_i = C^{\mathcal{OPT}}(I_i) \leq C^{\mathcal{OPT}}(I)$ for every $1 \leq i \leq k$, as the cost of serving all clients cannot be less than serving a single client. Plugging (2.1.2) back into (2.1.1) gives:

$$C^{\mathcal{SEQ}}(I) \leq k \cdot C^{\mathcal{OPT}}(I),$$

which implies the theorem. ■

Notice that by definition $\mathcal{SEQ}$ is a lazy algorithm. In Theorem 2.19, we prove that any lazy online algorithm for the uniform k-client problem with the total distance cost function is strictly k-competitive. Hence, by Theorem 2.19, $\mathcal{SEQ}$ is strictly k-competitive. However, due to the exclusive proof for $\mathcal{SEQ}$, the proof of Theorem 2.1, we were easily able to find a family of input instances in which the algorithm performs poorly. Using these input instances we now show that on any uniform metric space with at least $k+1$ points the upper bound above is tight, meaning the competitive ratio of $\mathcal{SEQ}$ is k.

Theorem 2.2 (Lower Bound for $\mathcal{SEQ}$ on $\mathcal{U}_{k+1}$)**.** *Let $\mathcal{U}_{k+1}$ be some uniform metric space with $k+1$ points. The competitive ratio of $\mathcal{SEQ}$ for the k-client problem on $\mathcal{U}_{k+1}$ is at least k.*

Proof. Let n be an arbitrary even positive integer. Consider an input instance I for the lower bound illustrated in Figure 2.1. In this instance, all the k client sequence have a fixed length n. That is, $n_i = n$ for all $1 \leq i \leq k$. Also, the clients are labeled according to the order that $\mathcal{SEQ}$ selected.

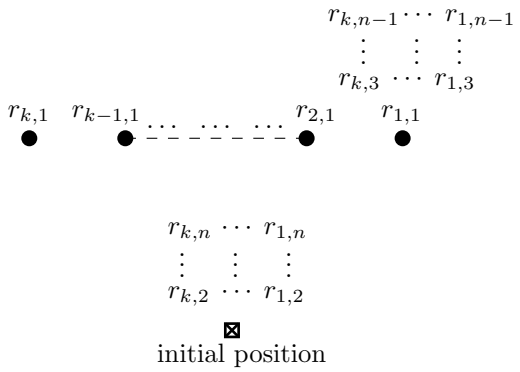

Figure 2.1: Lower Bound Instance for $\mathcal{SEQ}$ on $\mathcal{U}_{k+1}$.

The optimal solution for this instance is to move the server to the first request of each client, according to some order of the clients. In other words, the order in which the server moves to the first requests of the clients is not important. After that, the server must move alternately between the initial point and the other requested point, in order to serve all the other requests of all clients at once. This leads to an optimal cost of $n + (k-1)$, i.e. $C^{\mathcal{OPT}}(I) = n + (k-1)$. However, $\mathcal{SEQ}$ will move the server to serve all the requests of client 1, then move to serve all the requests of client 2, and so on. This results in a total cost of kn, so $C^{\mathcal{SEQ}}(I) = kn$. Therefore,

$$C^{\mathcal{SEQ}}(I) \geq kn = k\,(n + (k-1) - (k-1)) = k \cdot C^{\mathcal{OPT}}(I) - k\,(k-1)\,.$$

Moreover, $C^{\mathcal{OPT}}(I) = n + (k-1)$ can be made arbitrary large by increasing enough the integer n. Hence, by Lemma 1.9, the competitive ratio of $\mathcal{SEQ}$ for the k-client problem on $\mathcal{U}_{k+1}$ is at least k. ■

The $\mathcal{SEQ}$ algorithm is quite simple, since it always considers only one client sequence. However, this strategy may result in wasteful moves and is also unfair to the last client, who may have to wait a very long time before it can be served. Therefore, the question arises whether there is an algorithm that maintains fairness more and achieves a smaller competitive ratio. We now give an alternative deterministic strictly k-competitive algorithm that maintains fairness more. The algorithm is called Sorted Path ($\mathcal{SP}$).

Initially, the $\mathcal{SP}$ algorithm defines an order of the clients for breaking ties and an array $\mathcal{R}$ that contains all the outstanding client requests. The $\mathcal{SP}$ algorithm works in phases and at each phase, it first adds all the next client requests to $\mathcal{R}$ and then determines the movement path of the server at points in $\mathcal{R}$ in the following manner. The server will be moved to the point containing the most client requests in $\mathcal{R}$ until all the requests in $\mathcal{R}$ are served. In case of a tie, i.e. when there are at least 2 different points with the greatest amount of requests in $\mathcal{R}$, the server will be moved to the point containing the request of the client

with the lowest value label in the order of the clients until all the requests at those points are served. The algorithm terminates when there are no longer outstanding client requests, i.e. when $\mathcal{R}$ is empty at the time of determining the server's movement path. The $\mathcal{SP}$ algorithm is described in detail in Algorithm 2.2.

Algorithm 2.2 Sorted Path ($\mathcal{SP}$)

1: **function** MAIN(p_0, I)
2: **Input:** initial position of the server p_0
input instance I that consists of k client sequences starting at p_0
3: **Output:** output sequence
4: select an arbitrary order of the clients from 1 to k
5: $\mathcal{R} \leftarrow \{\}$ ▷ Initializing the array $\mathcal{R}$
6: `Add`($\mathcal{R}$) ▷ Adding all the next client requests to $\mathcal{R}$
7: **while** $|\mathcal{R}| \neq 0$ **do**
8: `DeterminePath`($\mathcal{R}$) ▷ Determinning the path and serving
9: `add`($\mathcal{R}$)
10: **end while**
11: **end function**

Function Add

12: **function** ADD($\mathcal{R}$)
13: **Input:** array of client requests $\mathcal{R}$
14: **for** $i \leftarrow 1$ to k **do**
15: **if** there is an outstanding request r_i **then**
16: add r_i to the array $\mathcal{R}$
17: **end if**
18: **end for**
19: **end function**

Function DeterminePath

20: **function** DETERMINEPATH($\mathcal{R}$)
21: **Input:** array of client requests $\mathcal{R}$
22: sort R by sequences of the same requests from the most common request to the least common request
23: **while** $|\mathcal{R}| \neq 0$ **do**
24: **if** there is no subsequent sequence of requests of the same length as the first sequence of requests in $\mathcal{R}$ **then**
25: let p be the point of the first sequence of request in $\mathcal{R}$ and move the server to p
26: `Remove`($p, \mathcal{R}$)
27: **else**
28: let p be the point of the sequence that has a request from the client with the lowest label value among all the sequences of the same length as the first sequence in $\mathcal{R}$ and move the server to p
29: `Remove`($p, \mathcal{R}$)
30: **end if**
31: **end while**
32: **end function**

Function Remove

33: **function** REMOVE($p, \mathcal{R}$)
34: **Input:** point p
array of client requests $\mathcal{R}$
35: serve all the requests of point p and remove them from $\mathcal{R}$
36: **end function**

Let us now prove that $\mathcal{SP}$ is strictly k-competitive for the k-client problem on any uniform metric space.

Theorem 2.3 (Upper Bound for $\mathcal{SP}$: Total Distance)**.** *The $\mathcal{SP}$ algorithm is strictly k-competitive for the k-client problem on any uniform metric space, where the cost function is the total distance.*

Proof. Given an input instance I that consists of requests in some uniform metric space, let the clients be ordered according to the order chosen by the $\mathcal{SP}$ algorithm. Let $\widetilde{n}_i$ denote the number of sequences of identical non-dummy requests in the set of requests of client i, $I_i \subseteq I$, for $1 \leq i \leq k$. Note that for any $1 \leq i \leq k$, $\widetilde{n}_i$ is exactly the optimal cost of serving the set of requests of client i, i.e. $\widetilde{n}_i = C^{\mathcal{OPT}}(I_i)$, since all the requests are in an uniform metric space. Denote $\widetilde{n}_{\max} = \max_{1 \leq i \leq k} \widetilde{n}_i$ and observe that

$$C^{\mathcal{OPT}}(I) \geq \widetilde{n}_{\max}, \tag{2.1.3}$$

since $\widetilde{n}_{\max}$ is exactly the optimal cost of serving the set of requests of the client of index $\arg\max_{1 \leq i \leq k} \widetilde{n}_i$ and it cannot be greater than the optimal cost of serving all the requests in I (due to the triangle inequality). However, in each phase, $\mathcal{SP}$ serves (at least) one request of from each client with an outstanding request at a cost no exceeding k, since the server can visit at most k different points. As a result, the number of phases is upper-bounded by $\widetilde{n}_{\max}$. Therefore, $C^{\mathcal{SP}}(I) \leq k \cdot \widetilde{n}_{\max}$. Plugging (2.1.3) into this inequality gives:

$$C^{\mathcal{SP}}(I) \leq k \cdot C^{\mathcal{OPT}}(I).$$

Hence the proof.

Notice that by definition $\mathcal{SP}$ is a lazy algorithm. Hence, by Theorem 2.19, $\mathcal{SP}$ is strictly k-competitive. We attempted to achieve a result regarding the quality of the performance of $\mathcal{SP}$ using the usual definition of competitive ratio, but we did not succeed. Yet, we managed to derive a lower bound $\frac{k+1}{2}$ on the strict competitive ratio of $\mathcal{SP}$ (so there is a gap of about a factor of 2 between the upper bound and the lower bound) for the k-client problem on any uniform metric space with at least $k+1$ points. Due to the exclusive proof for $\mathcal{SP}$, the proof of Theorem 2.3, we were able to find the lower bound input instance. ■

Theorem 2.4 (Lower Bound for $\mathcal{SP}$ on $\mathcal{U}_{k+1}$: Total Distance)**.** *Let $\mathcal{U}_{k+1}$ be some uniform metric space with $k+1$ points. The strict competitive ratio of $\mathcal{SP}$ for the k-client problem on $\mathcal{U}_{k+1}$ is at least $\frac{k+1}{2}$, where the cost function is the total distance.*

Proof. Without loss of generality, we assume that the order of the clients matches the order according to which $\mathcal{SP}$ will serve requests in case of ties. Let $\alpha_{\mathcal{SP}}$ denote the strict competitive ratio of $\mathcal{SP}$ for the k-client problem on $\mathcal{U}_{k+1}$. By the definition of a strict competitive ratio (1.1.1), it is sufficient to show that there exists an input instance I on $\mathcal{U}_{k+1}$ such that $\frac{C^{\mathcal{SP}}(I)}{C^{\mathcal{OPT}}(I)} \geq \frac{k+1}{2}$, in order to conclude that $\alpha_{\mathcal{SP}} \geq \frac{k+1}{2}$. Consider the input instance I illustrated

in Figure 2.2. In this instance, the length of each client sequence is equal to the label of the client according to order of the clients. In other words, $n_i = i$ for all $1 \leq i \leq k$.

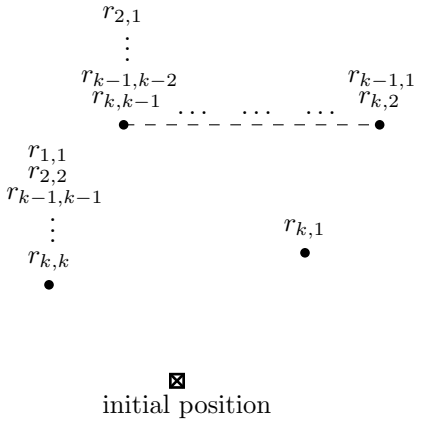

Figure 2.2: Lower Bound Instance for $\mathcal{SP}$ on $\mathcal{U}_{k+1}$.

The optimal solution for I is to move the server to the first request of client k and then to its subsequent requests, so that the server serves also all the requests from the other clients. That is, the server serves the requests according to the reverse order of the selected order by $\mathcal{SP}$ for breaking ties: it starts with client k and ends with client 1. This yields an optimal cost of k, so $C^{\mathcal{OPT}}(I) = k$. However, since the client requests are located at different points, $\mathcal{SP}$ will initially move the server to the first request of each client, according to its selected order of the clients for breaking ties. As a result, all the requests of client 1 are served and the cost of this move is k. After that, the next client requests are located at different points, so $\mathcal{SP}$ will move the server again, according to its chosen order of clients. Consequently, all the requests of clients 2 are served and the cost of this move is $k-1$. All of $\mathcal{SP}$'s next moves are determined in a similar manner, resulting in a total cost of

$$k + (k-1) + \ldots + 1 = \sum_{i=1}^{k} i = \frac{k(k+1)}{2}.$$

Therefore, $C^{\mathcal{SP}}(I) = \frac{k(k+1)}{2}$ and $\frac{C^{\mathcal{SP}}(I)}{C^{\mathcal{OPT}}(I)} = \frac{k+1}{2}$. In particular, we have shown that $\frac{C^{\mathcal{SP}}(I)}{C^{\mathcal{OPT}}(I)} \geq \frac{k+1}{2}$ and hence $\alpha_{\mathcal{SP}} \geq \frac{k+1}{2}$. ■

2.1.1.1 The Greedy Policy is No More Beneficial

A greedy natural online algorithm for the uniform varient of the k-client problem is to service as many pending requests as possible at once, if the algorithm has to move the server. This algorithm is called Common Point First ($\mathcal{CPF}$).

Initially, the $\mathcal{CPF}$ algorithm defines an order of the clients for breaking ties. The $\mathcal{CPF}$ algorithm works in phases and at each phase, it moves the server to the point containing the most pending client requests and services requests corresponding to that point as long as there are requests. In case of a tie, i.e. when there are at least 2 different points with the greatest amount of pending client requests, the $\mathcal{CPF}$ algorithm will move to the point containing the request of the client with the lowest value label in the order of the clients and services requests corresponding to that point as long as there are requests. The algorithm terminates when there are no longer pending client requests.

In Theorem 2.19, we prove that any lazy online algorithm for the uniform k-client problem with the total distance cost function is strictly k-competitive. In this section, we show that the greedy policy of $\mathcal{CPF}$ is no more beneficial than other lazy algorithms. More precisely, we show that the competitive ratio of $\mathcal{CPF}$ is k, if the uniform metric space has at least $k+1$ points. Note that by Definition 1.13, $\mathcal{CPF}$ is a lazy algorithm for the uniform k-client problem. Hence, by Theorem 2.19, $\mathcal{CPF}$ is a strictly k-competitive algorithm for the uniform k-client problem. So to obtain the desired result, it is sufficient to show that the competitive ratio of $\mathcal{CPF}$ is at least k, if the uniform metric space has at least $k+1$ points.

Theorem 2.5 (Lower Bound for $\mathcal{CPF}$ on $\mathcal{U}_{k+1}$)**.** *Let $\mathcal{U}_{k+1}$ be some uniform metric space with $k+1$ points. The competitive ratio of $\mathcal{CPF}$ for the k-client problem on $\mathcal{U}_{k+1}$ is at least k.*

Proof. Without loss of generality, we assume that the order of the clients corresponds to the order chosen by $\mathcal{CPF}$ for breaking ties. Let n be an arbitrary even positive integer. Consider an input instance I defined as the input instance from the proof of Theorem 2.2. The optimal solution for I is to move the server to the first request of each client, according to some order of the clients. In other words, the order in which the server moves to the first requests of the clients is not matter. After that, the server must move alternately between the initial point and the other requested point, in order to serve all the other requests of all clients at once. This leads to an optimal cost of $n+(k-1)$, i.e. $C^{\mathcal{OPT}}(I) = n+(k-1)$. However, at each phase during the execution of $\mathcal{CPF}$ on I, there is a case of a tie and $\mathcal{CPF}$ behaves as $\mathcal{SEQ}$. Hence, $\mathcal{CPF}$ will move the server to serve all the requests of client 1, then move to serve all the requests of client 2, and so on. This results in a total cost of kn, so $C^{\mathcal{CPF}}(I) = kn$. Therefore,

$$C^{\mathcal{CPF}}(I) \geq kn = k\,(n+(k-1)-(k-1)) = k \cdot C^{\mathcal{OPT}}(I) - k\,(k-1)\,.$$

Moreover, $C^{\mathcal{OPT}}(I) = n+(k-1)$ can be made arbitrary large by increasing enough the integer n. Hence, by Lemma 1.9, the competitive ratio of $\mathcal{CPF}$ for the k-client problem on $\mathcal{U}_{k+1}$ is at least k. ■

2.1.2 The Average Completion Time Cost Function

Throughout this section, only the average completion time cost function is considered. We now prove that $\mathcal{SP}$ is strictly k-competitive for the k-client problem on any uniform metric space. We improve the obtained result on the upper bound of the competitive ratio in [1], in which strictly $(2k-1)$-competitive algorithms are given. Notice that by definition $\mathcal{SP}$ is a lazy algorithm. In Theorem 2.20, we prove that any lazy online algorithm for the uniform k-client problem with the average completion time cost function is strictly k-competitive. Hence, by Theorem 2.20, $\mathcal{SP}$ is strictly k-competitive. However, we believe that due to this excursive proof for $\mathcal{SP}$, one can find input instances to derive lower bounds on its competitive ratio. We first show the following claim, which will be used later.

Claim 2.6. Let $I = \langle I_1, \ldots, I_k \rangle$ be an input instance for the k-client problem on any uniform metric space. Denote by $\widetilde{n}_i$the number of sequences of identical non- dummy requests in the set of requests of client i, $I_i \subseteq I$, for $1 \leq i \leq k$. Then, $\frac{1}{k}\sum_{i=1}^{k} \widetilde{n}_i \leq C^{\mathcal{OPT}}(I)$ for the average completion cost function.

Proof. Note that for any $1 \leq i \leq k$, $\mathcal{OPT}$cannot complete client i before $\widetilde{n}_i$, which is exactly the time it takes to complete client i if there are no other clients. Therefore $C^{\mathcal{OPT}}(I)$, the average completion time obtained by $\mathcal{OPT}$, is lower bounded by $\frac{1}{k}\sum_{i=1}^{k} \widetilde{n}_i$. ■

Theorem 2.7 (Upper Bound for $\mathcal{SP}$: Average Completion Time)**.** *The $\mathcal{SP}$ algorithm is strictly k-competitive for the k-client problem on any uniform metric space, where the cost function is the average completion time.*

Proof. Given an input instance $I = \langle I_1, \ldots, I_k \rangle$ that consists of requests in some uniform metric space, let the clients be labeled according to the order chosen by the $\mathcal{SP}$ algorithm. Let $\widetilde{n}_i$ denote the number of sequences of identical non-dummy requests in the set of requests of client i, for $1 \leq i \leq k$. We bound $C^{\mathcal{SP}}(I)$ by bounding the completion time of each client of I. For $1 \leq i \leq k$, the completion time of client i is at most $i \cdot \widetilde{n}_i + (k-i) \cdot (\widetilde{n}_i - 1) = k \cdot \widetilde{n}_i - (k-i)$. This bound considers the worst-case scenario. In this scenario, at each phase of $\mathcal{SP}$ all the k clients have to be served. Also, the requests of the clients are located at different points during each phase, so that the server does not process any new requests from each client that are not in its current position. As a result, for any $1 \leq j \leq i$, the server visits $\widetilde{n}_i$ points to service only client j. In addition, for any $i+1 \leq j \leq k$, the server visits $\widetilde{n}_i - 1$ points to service only client j. In total, we obtain the upper bound on the completion time of

each client of I. Therefore, we obtain the following upper bound on $C^{\mathcal{SP}}(I)$.

$$C^{\mathcal{SP}}(I) \leq \left(\frac{1}{k}\right) \cdot \left(\sum_{i=1}^{k} (k \cdot \widetilde{n}_i - (k-i))\right) =$$
$$\left(\frac{1}{k}\right) \cdot \left(\sum_{i=1}^{k} k \cdot \widetilde{n}_i - \sum_{i=1}^{k} (k-i)\right) =$$
$$k \cdot \left(\frac{1}{k} \sum_{i=1}^{k} \widetilde{n}_i\right) - \frac{1}{k} \cdot \sum_{i=1}^{k} (k-i) =$$
$$k \cdot \left(\frac{1}{k} \sum_{i=1}^{k} \widetilde{n}_i\right) - \frac{1}{k} \cdot \frac{k(k-1)}{2} =$$
$$k \cdot \left(\frac{1}{k} \sum_{i=1}^{k} \widetilde{n}_i\right) - \frac{k-1}{2} \leq k \cdot \left(\frac{1}{k} \sum_{i=1}^{k} \widetilde{n}_i\right),$$

where the last inequality holds for any $k \geq 1$. From Claim 2.6 we can conclude

$$C^{\mathcal{SP}}(I) \leq k \cdot C^{\mathcal{OPT}}(I).$$

Hence the proof.

We attempted to achieve a result regarding the quality of the performance of $\mathcal{SP}$ using the usual definition of competitive ratio, but we did not succeed. Yet, we have managed to derive a lower bound of $\frac{1}{3}\left(k - \frac{1}{2} + \frac{9}{4k}\right)$ on the strict competitive ratio of $\mathcal{SP}$ (so there is a gap of about a factor of 3 between the upper bound and the lower bound) for the k-client problem on any uniform metric space with at least $k+1$ points. ■

Theorem 2.8 (Lower Bound for $\mathcal{SP}$ on $\mathcal{U}_{k+1}$: Average Completion Time)**.** *Let $\mathcal{U}_{k+1}$ be some uniform metric space with $k+1$ points. The strict competitive ratio of $\mathcal{SP}$ for the k-client problem on $\mathcal{U}_{k+1}$ is at least $\frac{1}{3}\left(k - \frac{1}{2} + \frac{9}{4k}\right)$, where the cost function is the average completion time.*

Proof. Without loss of generality, we assume that the order of the clients corresponds to the order chosen by $\mathcal{SP}$ for breaking ties. Let $\alpha_{\mathcal{SP}}$ denote the strict competitive ratio of $\mathcal{SP}$ for the k-client problem on $\mathcal{U}_{k+1}$. By the definition of a strict competitive ratio (1.1.1), it is sufficient to show that there exists an input instance I on $\mathcal{U}_{k+1}$ such that

$$\frac{C^{\mathcal{SP}}(I)}{C^{\mathcal{OPT}}(I)} \geq \frac{1}{3}\left(k - \frac{1}{2} + \frac{9}{4k}\right), \tag{2.1.4}$$

in order to conclude that $\alpha_{\mathcal{SP}} \geq \frac{1}{3}\left(k - \frac{1}{2} + \frac{9}{4k}\right)$. Consider the input instance I from the proof of Theorem 2.4. Given the input instance I, let $\mathcal{ALG}$ be an algorithm which services request sequentially, according to the reverse order of

the selected order by $\mathcal{SP}$ for breaking ties. By optimality of $\mathcal{OPT}$, we have $C^{\mathcal{ALG}}(I) \geq C^{\mathcal{OPT}}(I)$ and thus

$$\frac{C^{\mathcal{SP}}(I)}{C^{\mathcal{ALG}}(I)} \leq \frac{C^{\mathcal{SP}}(I)}{C^{\mathcal{OPT}}(I)}. \tag{2.1.5}$$

Therefore, from (2.1.4) and (2.1.5), it is sufficient to show that $\frac{C^{\mathcal{SP}}(I)}{C^{\mathcal{ALG}}(I)} \geq \frac{1}{3}\left(k - \frac{1}{2} + \frac{9}{4k}\right)$, in order to conclude that $\alpha_{\mathcal{SP}} \geq \frac{1}{3}\left(k - \frac{1}{2} + \frac{9}{4k}\right)$. The completion time of each of the k clients obtained by $\mathcal{ALG}$ is k and thus the average completion time obtained by $\mathcal{ALG}$ is also k. However, the completion time of client 1 obtained by $\mathcal{SP}$ is 1 and the completion time of client i obtained by $\mathcal{SP}$ is $1 + \sum_{j=1}^{i-1} k - (j-1)$, for $2 \leq i \leq k$. Hence,

$$\begin{aligned}
C^{\mathcal{SP}}(I) &= \frac{\sum_{i=1}^{k} 1 + \sum_{i=2}^{k}\sum_{j=1}^{i-1}(k-(j-1))}{k} = \frac{k}{k} + \frac{\sum_{i=2}^{k}\frac{i-1}{2}\cdot(k+k-(i-2))}{k} \\
&= 1 + \frac{\sum_{i=2}^{k}\frac{i-1}{2}\cdot(2k-(i-2))}{k} \underbrace{=}_{l \leftrightarrow i-1} 1 + \frac{\sum_{l=1}^{k-1}\frac{l}{2}\cdot(2k-(l-1))}{k} \\
&= 1 + \frac{k\sum_{l=1}^{k-1} l}{k} - \frac{1}{2}\cdot\frac{\sum_{l=1}^{k-1} l^2}{k} + \frac{1}{2}\cdot\frac{\sum_{l=1}^{k-1} l}{k} \\
&= 1 + \frac{k(k-1)}{2} - \frac{1}{2k}\cdot\frac{(k-1)k(2k-1)}{6} + \frac{1}{2k}\cdot\frac{k(k-1)}{2} \\
&= 1 + \frac{k(k-1)}{2} - \frac{(k-1)(2k-1)}{12} + \frac{k-1}{4},
\end{aligned}$$

where the second last equality uses the summation formula $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$. Thus, we have

$$\begin{aligned}
\frac{C^{\mathcal{SP}}(I)}{C^{\mathcal{ALG}}(I)} &= \frac{C^{\mathcal{SP}}(I)}{k} = \frac{1}{k} + \frac{k-1}{2} - \frac{(k-1)(2k-1)}{12k} + \frac{k-1}{4k} \\
&\geq \frac{1}{k} + \frac{k-1}{2} - \frac{k(2k-1)}{12k} + \frac{k-1}{4k} = \frac{1}{k} + \frac{k-1}{2} - \frac{2k-1}{12} + \frac{k-1}{4k} \\
&= \frac{1}{k} + \frac{k}{2} - \frac{1}{2} - \frac{k}{6} + \frac{1}{12} + \frac{1}{4} - \frac{1}{4k} = \frac{k}{3} - \frac{1}{6} + \frac{3}{4k} = \frac{1}{3}\left(k - \frac{1}{2} + \frac{9}{4k}\right).
\end{aligned}$$

So, we have shown that $\frac{C^{\mathcal{SP}}(I)}{C^{\mathcal{ALG}}(I)} \geq \frac{1}{3}\left(k - \frac{1}{2} + \frac{9}{4k}\right)$ and hence $\alpha_{\mathcal{SP}} \geq \frac{1}{3}\left(k - \frac{1}{2} + \frac{9}{4k}\right)$. ■

2.2 The Randomized Sequential Algorithm

We now consider the total distance cost function and the Randomized Sequential ($\mathcal{RSEQ}$) algorithm, which uniformly at random selects an order of the clients from 1 to k. Afterwards, it services all the requests of client 1, then all the requests of client 2, etc. Note that unlike the $\mathcal{SEQ}$ algorithm, in different

runs on a given input instance, different orders of the clients can be chosen randomly, so the cost of the $\mathcal{RSEQ}$ for this input instance is a random variable that depends on the chosen order of the clients. As in the $\mathcal{SEQ}$ algorithm, if the server passes over requests from other clients while serving some client, it will still serve them. The $\mathcal{RSEQ}$ is detailed in Algorithm 2.3.

Algorithm 2.3 The $\mathcal{RSEQ}$ Algorithm

Require: initial position of the server p_0
    k clients who each have a sequence of requests starting at p_0
select **uniformly at random** an order of the clients from 1 to k.
for $i \leftarrow 1$ to k **do**
    serve all the requests of the i-th client.
end for

In [1], the $\mathcal{RSEQ}$ algorithm was used to show that the achieved randomized lower bound of $\Omega(\log\log k)$ (according to the strict definition of competitive ratio) against oblivious adversaries is tight for the input distribution used in its proof. We now show that a random selection of the clients is not better than a concrete selection of the order of the clients. In particular, we prove that the competitive ratio of $\mathcal{RSEQ}$ against an adaptive online adversary ($\mathcal{ADON}$) is at least k on any uniform metric space with at least $k+1$ points. We also prove the same randomized lower bound for the competitive ratio when $\mathcal{RSEQ}$ is compared against an oblivious adversary ($\mathcal{OBL}$), but on any uniform metric space with at least $k+2$ points. For any uniform metric space with $k+1$ points $\mathcal{U}_{k+1}$, we show a randomized lower bound of $k - \frac{k-1}{k}$ on the competitive ratio of $\mathcal{RSEQ}$ against an oblivious adversary.

Theorem 2.9 (Lower Bound for $\mathcal{RSEQ}$ against $\mathcal{ADON}$)**.** *Let $\mathcal{U}_{k+1}$ be some uniform metric space with $k+1$ points. The competitive ratio of $\mathcal{RSEQ}$ against an adaptive online adversary for the k-client problem on $\mathcal{U}_{k+1}$ is at least k.*

Proof. Let n be an arbitrary even positive integer and let $p \in \mathcal{U}_{k+1}$. We fix an order of the clients from 1 to k and assume that the clients are ordered according to it. We define an adaptive online adversary $\mathsf{A}(p,n)$ against $\mathcal{RSEQ}$ as follows:

- The initial point of the server is p.
- The number of requests for each client is n, i.e. $n_i = n$ for $1 \leq i \leq k$.
- The adversary begins by planting the first request of client 1 through k in $\mathcal{U}_{k+1} \backslash \{p\}$ at different points.
- Suppose $\mathcal{RSEQ}$ selected client i' as the first client to service and let $p_{i'}$ denote the location of the first request of client j. For each $1 \leq i \leq k$ and $2 \leq j \leq n$, the adversary will place the j-th request of client i at p if j is an even number. Otherwise, the request will be located at $p_{i'}$.

- The online strategy of the adversary works in iterations. At each iteration, the adversary moves the server to service one request of each client from the set of non-served clients, according to the order of the clients. Notice that at each iteration, the server may service more than one request of the same client.

Let I denote the generated input instance by the adversary and let $\mathcal{I}$ denote the set of all input instances generated by adversary $\mathsf{A}(p,n)$. Note that I is a random variable that depends only on the random choice of $\mathcal{RSEQ}$ regarding which client it services first. Thus, the cost of the online strategy of adversary $\mathsf{A}(p,n)$, $C^{\mathcal{ADON}}(I)$, is also a random variable. Figure 2.1 illustrated I in case $\mathcal{RSEQ}$ services client 1 first. We have that $|\mathcal{I}| = k$. Moreover, the adversary determines the input only according to the first request of the first client served by $\mathcal{RSEQ}$ and $\mathcal{RSEQ}$ selects each client as its first client w.p. $\frac{1}{k}$. Thus, each input instance I is generated by the adversary w.p. $\frac{1}{k}$. The adversary generates the input instance such that $\mathcal{RSEQ}$ has to move the server to serve all the requests of its first client, then move to serve all the requests of its second client, and so on. As a result, the cost of $\mathcal{RSEQ}$ to service any input instance $I \in \mathcal{I}$ is kn. Therefore,

$$\mathbb{E}\left[C^{\mathcal{RSEQ}}(I)\right] = \sum_{I\in\mathcal{I}} \frac{1}{k} \cdot C^{\mathcal{RSEQ}}(I) = k \cdot \frac{1}{k} \cdot (kn) = kn.$$

However, for each $I \in \mathcal{I}$ the adversary first moves the server to the first request of each client, according to its selected order of the clients. Then, the adversary must move the server alternately between the initial point and the other requested point. Consequently, all non-first requests of all clients will be serviced in parallel. The obtained cost of the adversary is $n + (k-1)$, for each $I \in \mathcal{I}$. Hence,

$$\mathbb{E}\left[C^{\mathcal{ADON}}(I)\right] = \sum_{I\in\mathcal{I}} \frac{1}{k} \cdot C^{\mathcal{ADON}}(I) = k \cdot \frac{1}{k} \cdot (n + (k-1)) = n + (k-1).$$

Therefore, the expected cost of $\mathcal{RSEQ}$ is at least:

$$\mathbb{E}\left[C^{\mathcal{RSEQ}}(I)\right] \geq kn = k\left(n + (k-1) - (k-1)\right) = k \cdot \mathbb{E}\left[C^{\mathcal{ADON}}\right] - k\left(k-1\right).$$

Moreover, $\mathbb{E}\left[C^{\mathcal{ADON}}(I)\right] = n + (k-1)$ can be made arbitrary large by increasing enough the integer n. Hence, by Lemma 1.11, the competitive ratio of $\mathcal{RSEQ}$ against an adaptive online adversary for the k-client problem on $\mathcal{U}_{k+1}$ is at least k.

■

Theorem 2.10 (Lower Bound for $\mathcal{RSEQ}$ against $\mathcal{OBL}$)**.** *Let $\mathcal{U}_{k+2}$ be some uniform metric space with $k+2$ points. The competitive ratio of $\mathcal{RSEQ}$ against an oblivious adversary for the k-client problem on $\mathcal{U}_{k+2}$ is at least k.*

Proof. Again, we fix an order of the clients and assume that the clients are labeled according to it. Consider an oblivious adversary that chooses an input instance I depicted in Figure 2.3. In this instance, all the k clients have a fixed length n where n is an even number. Namely, $n_i = n$ for all $1 \leq i \leq k$. The first requests of the clients are located at k different points in $\mathcal{U}_{k+2}$. Also, for each $1 \leq i \leq k$ and $2 \leq j \leq n$, the j-th request of client i is placed at the initial point if j is an even number. All other requests are located at the remaining point, which is the point that does not contain a first request from any client and that is not the initial point.

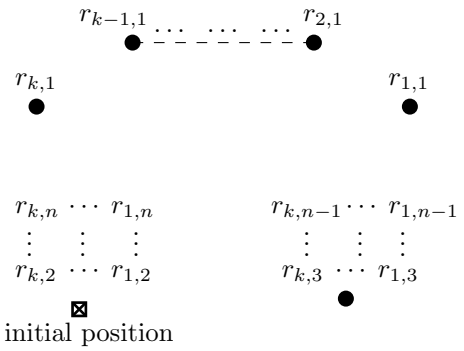

Figure 2.3: Lower Bound Instance Chosen by $\mathcal{OBL}$ for $\mathcal{RSEQ}$ on $\mathcal{U}_{k+2}$.

Here, the optimal solution for this instance is to move the server to the first requests of the clients, according to any order. After that, the server must move alternately between the initial point and the remaining requested point, in order to serve all the other requests of all clients at once. This leads to an optimal cost of $n + (k-1)$, so $C^{\mathcal{OPT}}(I) = n + (k-1)$. However, $\mathcal{RSEQ}$ works in the same manner on the instance I regardless of the selected order of clients. More precisely, $\mathcal{RSEQ}$ moves the server to serve all the requests of its first client, then move to serve all the requests of its second client, and so on. As a result, $C^{\mathcal{RSEQ}}(I) = kn$ w.p. 1 and thus $\mathbb{E}\left[C^{\mathcal{RSEQ}}(I)\right] = kn$. Therefore, the expected cost of $\mathcal{RSEQ}$ is at least:

$$\mathbb{E}\left[C^{\mathcal{RSEQ}}(I)\right] \geq kn = k\left(n + (k-1) - (k-1)\right) = k \cdot C^{\mathcal{OPT}}(I) - k\left(k-1\right).$$

Moreover, $C^{\mathcal{OPT}}(I) = n + (k-1)$ can be made arbitrary large by increasing enough the integer n. Hence, by Lemma 1.11, the competitive ratio of $\mathcal{RSEQ}$ against an oblivious adversary for the k-client problem on $\mathcal{U}_{k+2}$ is at least k. ■

Theorem 2.11 (Lower Bound for $\mathcal{RSEQ}$ against $\mathcal{OBL}$ on $\mathcal{U}_{k+1}$)**.** *Let $\mathcal{U}_{k+1}$ be some uniform metric space with $k+1$ points. The competitive ratio of $\mathcal{RSEQ}$ against an oblivious adversary for the k-client problem on $\mathcal{U}_{k+1}$ is at least $k - \frac{k-1}{k}$.*

Proof. Without loss of generality, assume that $k \geq 2$[1]. Consider some order

[1]Indeed, the problem is trivial for $k = 1$ and any online algorithm is already optimal and in particular the competitive ratio is at least $1 - \frac{1-1}{1} = 1$.

of the clients from 1 through k and let the clients be labeled according to it. Consider an oblivious adversary that chooses an input instance I defined as follows. All the k clients have a fixed length n where n is an even number. Namely, $n_i = n$ for all $1 \leq i \leq k$. The first requests of the clients are located at k different points in $\mathcal{U}_{k+1}$. Also, for each $1 \leq i \leq k$ and $2 \leq j \leq n$, the j-th request of client i is placed at the initial point if j is an even number. All other requests are located at the point where the first request of client 1 is located and denote this point by p_1. The optimal solution for I is to move the server to the first requests of the clients, according to any order. Then, the server must move alternately between the initial point and p_1, in order to serve all the other requests of all clients in parallel. This leads to an optimal cost of $n + (k-1)$ and therefore $C^{\mathcal{OPT}}(I) = n + (k-1)$. However, $C^{\mathcal{RSEQ}}(I)$ has 3 possible values, that are corresponding to whether client 1 was chosen first, was chosen second or was not chosen first or second by $\mathcal{RSEQ}$. Client 1 is chosen to be the first client that $\mathcal{RSEQ}$ services w.p. $\frac{1}{k}$. In this scenario, $\mathcal{RSEQ}$ will move the server to serve all the requests of client 1, then move to serve all the requests of its second client, etc, which results in a total cost of kn. Client 1 is chosen to be the second client that $\mathcal{RSEQ}$ services w.p. $\frac{k-1}{k} \cdot \frac{1}{k-1} = \frac{1}{k}$. In this scenario, $\mathcal{RSEQ}$ will move the server to serve all the requests of its first client, including the first $n-2$ requests of client 1. Then it move the server to serve the remaining 2 requests of client 1 and then continue to move the server so that the remaining client sequences are served sequentially, which leads to a total cost of $(k-1)\,n+2$. Client 1 is not chosen to be the first client or second client that $\mathcal{RSEQ}$ services w.p. $\frac{k-1}{k} \cdot \frac{k-2}{k-1} = \frac{k-2}{k}$. In this scenario, $\mathcal{RSEQ}$ will move the server to serve all the requests of its first client, including the first $n-2$ requests of client 1. Then it move the server to serve all the requests of its second client, including the remaining 2 requests of client 1 and then continue to move the server so that the remaining client sequences are served sequentially, resulting in a total cost of $(k-1)\,n$. Hence,

$$\begin{aligned}
\mathbb{E}\left[C^{\mathcal{RSEQ}}(I)\right] &= kn \cdot \frac{1}{k} + ((k-1)\,n+2) \cdot \frac{1}{k} + (k-1)\,n \cdot \frac{k-2}{k} \\
&= \left(1 + \frac{k-1}{k} + \frac{(k-1)\,(k-2)}{k}\right) n + \frac{2}{k} \\
&= \left(\frac{k + (k-1)\,(1+k-2)}{k}\right) n + \frac{2}{k} = \left(\frac{k + (k-1)^2}{k}\right) n + \frac{2}{k} \\
&= \left(\frac{k + k^2 - 2k + 1}{k}\right) n + \frac{2}{k} = \left(k - \frac{k-1}{k}\right) n + \frac{2}{k}.
\end{aligned}$$

Therefore, the expected cost of $\mathcal{RSEQ}$ is at least:

$$\mathbb{E}\left[C^{\mathcal{RSEQ}}(I)\right] \geq \left(k - \frac{k-1}{k}\right) n + \frac{2}{k} = k\left(n + (k-1) - (k-1) - \frac{k-1}{k}\right) + \frac{2}{k}$$
$$= k \cdot C^{\mathcal{OPT}} - k(k-1) - (k-1) + \frac{2}{k} = k \cdot C^{\mathcal{OPT}}(I) - (k+1)(k-1) + \frac{2}{k}$$
$$= k \cdot C^{\mathcal{OPT}}(I) - \left(k^2 - 1 - \frac{2}{k}\right).$$

In addition, $C^{\mathcal{OPT}}(I) = n + (k-1)$ can be made arbitrary large by increasing enough the integer n. Since the inequality $k^2 - 1 - \frac{2}{k} \geq 0$ holds for each $k \geq 2$, we obtain by Lemma 1.11 that the competitive ratio of $\mathcal{RSEQ}$ against an oblivious adversary for the k-client problem on $\mathcal{U}_{k+1}$ is at least $k - \frac{k-1}{k}$. ■

2.3 A Deterministic Lower Bound

In this section, we prove that there is no deterministic algorithm with a competitive ratio lower than $\frac{\lfloor \log k \rfloor}{2} + 1 = \Omega(\log k)$ for the k-client problem with the total distance cost function on any uniform metric space with at least $2 \cdot 2^{\lfloor \log k \rfloor} = 2^{\lfloor \log k \rfloor + 1}$ points.

We first consider only k values that are powers of 2 and then we derive a general result for all $k \geq 1$. Let k' be a power of 2, i.e. $k' = 2^g$ for some integer $g \geq 0$. Let $\mathcal{U}_{2k'}$ be some uniform metric space with $2k'$ points. Let us define an adversary $\mathsf{ADV}(k', m)$ that constructs k' input sequences of the clients on $\mathcal{U}_{2k'}$, given an integer $m \geq 1$. In order to define the adversary, we fix an order of the clients from 1 to k and assume the clients are labeled according to it.

Definition 2.12 (Adversary $\mathsf{ADV}(k', m)$)**.** Given an order of the clients from 1 to k' and an integer $m \geq 1$, the adversary $\mathsf{ADV}(k', m)$ generates k' input sequences of the clients on $\mathcal{U}_{2k'}$ as follows:

- The adversary selects an arbitrary point $s \in \mathcal{U}_{2k'}$ as the initial point of the server.
- The adversary plants the first request of client 1 through k' in $\mathcal{U}_{2k'} \setminus \{s\}$ at different points. Let q_i denote the point where the first request of client i is located, for all $1 \leq i \leq k'$.
- The adversary plants the second request of client 1 through k' in $\mathcal{U}_{2k'} \setminus \{q_1, \ldots, q_{k'}\}$ at different points. Let w_i denote the point where the second request of client i is located, for all $1 \leq i \leq k'$. Note that one of the points in $\{w_1, \ldots, w_{k'}\}$ must be the initial point s. Without loss of generality, assume that $w_1 = s$.
- For each $1 \leq i \leq k'$ and $3 \leq j \leq m$, the adversary will place the j-th request of client i at q_i if j is an odd number. Otherwise, the request will be located at w_i.

- From now on, the adversary reacts only if there is a client with no outstanding request. If there is such client i, let n_i be the current length of its input sequence. The adversary can *concatenate* a particular client (see (a)) with input sequence of length n_i to client i by setting the $(n_i + h)$-th request of client i to be at the same point as the h-th request of the other client, for $1 \leq h \leq n_i$. The adversary reacts as follows.
  (a) If there is a client with a terminated input sequence of length n_i who has not been yet concatenated to any client, let j be its label according to the order of the clients. The adversary concatenates client j to client i.
  (b) If there is no such client j, the adversary terminates the input sequence of client i in $\mathcal{U}_{2k'}$.

We now prove the following result regarding the competitive ratio of any deterministic online algorithm for the problem with $k^{'}$ clients on $\mathcal{U}_{2k'}$.

Lemma 2.13. *Let $k^{'}$ be a power of 2, i.e. $k^{'} = 2^g$ for some integer $g \geq 0$. The competitive ratio of any deterministic online algorithm for the problem with $k^{'}$ clients on $\mathcal{U}_{2k'}$ is at least $\frac{\log k^{'}}{2} + 1$.*

Proof. Let $\mathcal{A}$ be some deterministic online algorithm for the problem with $k^{'}$ clients and let m be an arbitrary positive integer. Fix an order of the clients from 1 to $k^{'}$ and assume the clients are labeled according to it. We use adversary $\mathsf{ADV}(k^{'}, m)$ (see Definition 2.12) and denote by $I^{'}$ the input instance generated by $\mathsf{ADV}(k^{'}, m)$ during the execution of $\mathcal{A}$.

We first bound the cost of the online algorithm $\mathcal{A}$. Notice that from the construction of $\mathsf{ADV}(k^{'}, m)$ it follows that the online algorithm $\mathcal{A}$ can service at most one request, at any time during its execution. Indeed, at any time during the execution of $\mathcal{A}$, the outstanding requests of the clients are located at different point and thus $\mathcal{A}$ cannot service more than one request. As a result, the online cost incurred by $\mathcal{A}$ must be the number of requests in instance $I^{'}$. This input instance has the following properties. The first m requests of each two of the $k^{'}$ clients are located at different points, no client has more than one request at any point, and the first request of each client of the k client is not located at the initial point of the server. From these properties since each client can be concatenated by $\mathsf{ADV}(k^{'}, m)$ to only one client, it follows that there is there is a single client with $k^{'}m$ requests, and for each $0 \leq i \leq \log k^{'} - 1$, there are $\frac{k^{'}}{2^{i+1}}$ clients with $2^i m$ requests. Putting all together, we have

$$C^{\mathcal{A}}(I^{'}) = k^{'}m + \sum_{i=0}^{\log k^{'}-1} \frac{k^{'}}{2^{i+1}} \cdot 2^i m = k^{'}m + k^{'}m \sum_{i=0}^{\log k^{'}-1} \frac{1}{2}$$
$$= \left(\frac{\log k^{'}}{2} + 1\right) k^{'}m. \quad (2.3.1)$$

We now show that the cost of an optimal offline algorithm for instance $I^{'}$ is $k^{'}m$. This cost is obtained by serving the client with $k^{'}m$ requests, along with the requests of other clients. This solution obtains the optimal cost, as the optimal cost cannot be reduced by having more client to serve. Thus, $C^{\mathcal{OPT}}(I^{'}) = k^{'}m$. Plugging this equality back to (2.3.1) gives:

$$C^{\mathcal{A}}(I^{'}) \geq \left(\frac{\log k^{'}}{2} + 1\right) \cdot C^{\mathcal{OPT}}(I^{'}).$$

Furthermore, $C^{\mathcal{OPT}}(I^{'}) = k^{'}m$ can be made arbitrary large by increasing enough the integer m. Hence, by Lemma 1.9, the competitive ratio of $\mathcal{A}$ for the problem with $k^{'}$ clients on $\mathcal{U}_{2k'}$ is at least $\frac{\log k^{'}}{2} + 1$. ■

Lemma 2.13 directly gives us the desired result.

Theorem 2.14 (Deterministic Lower Bound)**.** *Let $k = k^{'} + r$ where $k^{'} = 2^{\lfloor \log k \rfloor}$[2] and $0 \leq r \leq 2^{\lfloor \log k \rfloor} - 1$. The competitive ratio of any deterministic online algorithm for the k-client problem on $\mathcal{U}_{2k'}$ is at least $\frac{\lfloor \log k \rfloor}{2} + 1 = \Omega(\log k)$.*

Proof. Let $\mathcal{A}$ be some deterministic online algorithm for the k-client problem and let m be an arbitrary positive integer. Fix an order of the clients from 1 to k and assume that the clients are labeled according to it. Consider the following input instance $I = \langle I_1, \ldots, I_k \rangle$. The sequences of client 1 through client $k^{'}$, i.e. $I_1, \ldots, I_{k'}$, are generated using the construction of adversary $\mathsf{ADV}(k^{'}, m)$ (see Definition 2.12). The remaining r sequences are exact copies of I_1. Notice that algorithm $\mathcal{A}$ for the k-client problem on instance I collapses into the $k^{'}$-client problem on instance $I^{'} = \langle I_1, \ldots, I_{k'} \rangle$. From Lemma 2.13, we have that the competitive ratio of $\mathcal{A}$ on $\mathcal{U}_{2k'}$ is at least $\frac{\log k^{'}}{2} + 1 = \frac{\lfloor \log k \rfloor}{2} + 1$. Therefore, the competitive ratio of $\mathcal{A}$ for the k-client problem on $\mathcal{U}_{2k'}$ is at least $\frac{\lfloor \log k \rfloor}{2} + 1 = \Omega(\log k)$. ■

2.4 Randomized Lower Bounds

2.4.1 A Constant Lower Bound

In this section, we show that there is no randomized algorithm with a competitive ratio lower than $1 + \frac{2-1}{2^2} = 1.25$ for the k-client problem with $k \geq 2$ and the total distance cost function on any uniform metric space with at least 4 points. To do this, we first prove the following lemma.

Lemma 2.15. *Let $\mathcal{U}_{2k}$ be some uniform metric space with $2k$. The competitive ratio of any randomized online algorithm against an oblivious adversary for the k-client problem with the total cost function on $\mathcal{U}_{2k}$ is at least $1 + \frac{k-1}{k^2}$.*

[2] Namely, $k^{'}$ is the most close power of 2 to k that is not greater than k.

Proof. Let $\mathcal{A}$ be some online randomized algorithm for the k-client problem and let m be an arbitrary positive integer. Fix an order of the clients from 1 to k and assume the clients are labeled according to it. Consider an oblivious adversary $\mathsf{A}(k, m)$ that generates an input instance I as follows.

- The adversary selects an arbitrary point $s^{'} \in \mathcal{U}_{2k'}$ as the initial point of the server.
- The adversary plants the first request of client 1 through k in $\mathcal{U}_{2k} \backslash \{s\}$ at different points. Let q_i denote the point where the first request of client i is located, for all $1 \leq i \leq k$.
- The adversary plants the second request of client 1 through k in $\mathcal{U}_{2k} \backslash \{q_1, \ldots, q_k\}$ at different points. Let w_i denote the point where the second request of client i is located, for all $1 \leq i \leq k$. Note that one of the points in $\{w_1, \ldots, w_k\}$ must be the initial point s. Without loss of generality, assume that $w_1 = s$.
- Denote the set $\{1, \ldots, k\}$ by $[k]$. For each $i \in [k]$ and $3 \leq j \leq m$, the adversary will place the j-th request of client i at q_i if j is an odd number. Otherwise, the request will be located at w_i.
- Denote by p_i the probability that client i is the client with the last sequence of requests of which $\mathcal{A}$ serves the m-th request, for all $1 \leq i \leq k$. Let $i^{'} = \arg\max_{1 \leq i \leq k} p_i$. If there are several such clients, consider one of them. Since the maximum probability in $\{p_1, , \ldots, p_k\}$ cannot be lower than the average of the probabilities, we have $p_{i'} \geq \frac{p_1 + \cdots + p_k}{k} = \frac{1}{k}$.
- For all $i \in [k] \backslash \{i^{'}\}$, the number of requests of client i is m, i.e. $n_i = m$. Furthermore, the number of requests of client $i^{'}$ is km, i.e. $n_{i'} = km$.
- Define:

$$l_i = \begin{cases} i & \text{if } i < i^{'} \\ i - 1 & \text{if } i > i^{'} \end{cases}$$

  for all $i \in [k] \backslash \{i^{'}\}$. For each $i \in [k] \backslash \{i^{'}\}$ and $1 \leq j \leq m$, the $(l_i \cdot m + j)$-th request of client $i^{'}$ will be located at the same point as request $r_{i,j}$. See Figure 2.4 for more illustration.

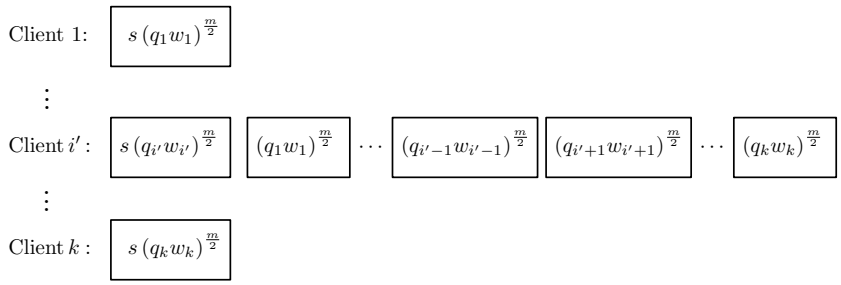

Figure 2.4: Randomized Lower Bound Instance Generated by $\mathsf{A}(s,m)$ on $\mathcal{U}_{2k}$.

The optimal solution for this instance is to serve the requests of client $i^{'}$, along with the requests of other clients. The obtained optimal cost is km, i.e. $C^{\mathcal{OPT}}(I) = km$. However, for this instance, algorithm $\mathcal{A}$ terminates serving the sequence of requests of client $i^{'}$ last w.p. at least $\frac{1}{k}$. In this scenario, the obtained cost of $\mathcal{A}$ is necessarily $(k-1)\,m + km = (2k-1)\,m$. On the other hand, client $i^{'}$ is not the client with the sequence of request served last by $\mathcal{A}$ w.p. at most $1 - \frac{1}{k} = \frac{k-1}{k}$. In such cases, the obtained cost of $\mathcal{A}$ is at least km (and at most $(k-2)\,m + km = (2k-2)m$). Therefore, the expected cost of algorithm $\mathcal{A}$ is at least:

$$\mathbb{E}\left[C^{\mathcal{A}}(I)\right] \geq \frac{1}{k} \cdot (2k-1)\,m + \frac{k-1}{k} \cdot km = \left(2 - \frac{1}{k} + k - 1\right) m$$
$$= \left(k + 1 - \frac{1}{k}\right) m = \left(1 + \frac{1}{k} - \frac{1}{k^2}\right) km =$$
$$\left(1 + \frac{k-1}{k^2}\right) km \geq \left(1 + \frac{k-1}{k^2}\right) \cdot C^{\mathcal{OPT}}(I).$$

Moreover, $C^{\mathcal{OPT}}(I) = km$ can be made arbitrary large by increasing enough the integer m. Hence, by Lemma 1.11, the competitive ratio of $\mathcal{A}$ against an oblivious adversary for the k-client problem on $\mathcal{U}_{2k}$ is at least $1 - \frac{k-1}{k^2}$. ■

Now, it is easy to verify that $\min_{k\geq 1} 1 + \frac{k-1}{k^2} = 1.25$ and that this minimum is reached by setting $k = 2$. Thus, one can improve the randomized lower bound for $k > 2$ achieved in the above lemma using the case of $k = 2$ in the above lemma and obtain the desired result.

Theorem 2.16 (Constant Randomized Lower Bound)**.** *The competitive ratio of any randomized online algorithm against an oblivious adversary for the k-client problem with $k \geq 2$ and the total cost function is at least 1.25 on $\mathcal{U}_4$.*

Proof. Let $\mathcal{A}$ be some randomized online algorithm for the k-client problem with $k \geq 2$ on $\mathcal{U}_4$ and let m be an arbitrary positive integer. Fix an order of the clients from 1 to k and consider the following input instance $I = \langle I_1, \ldots, I_k \rangle$. The sequences of client 1 and client 2, i.e. I_1, I_2, are generated using the construction

of adversary $\mathsf{A}(2, m)$ from the proof of Lemma 2.15. The remaining $k-2$ sequences are exact copies of I_1. Note that algorithm $\mathcal{A}$ for the k-client problem on instance I collapses into the 2-client problem on instance $\widetilde{I} = \langle I_1, I_2 \rangle$. From the proof of Lemma 2.15, we have that the competitive ratio of $\mathcal{A}$ on $\mathcal{U}_4$ is at least $1 + \frac{2-1}{2^2} = 1.25$. Therefore, the competitive ratio of $\mathcal{A}$ for the k-client problem on $\mathcal{U}_4$ is at least 1.25. ■

2.4.2 A Lower Bound of $\Omega(\log \log k)$

In this section, we prove that there is no randomized algorithm with a competitive ratio lower than $\Omega(\log \log k)$ for the k-client problem with the total distance cost function on any uniform metric space with at least $2 \cdot 2^{\lfloor \log k \rfloor} = 2^{\lfloor \log k \rfloor + 1}$ points.

As in Section 2.3, we first consider only k values that are powers of 2 and then we obtain a general result for all $k \geq 1$. Let $k^{'}$ be a power of 2, i.e. $k^{'} = 2^g$ for some integer $g \geq 0$. Let $\mathcal{U}_{2k^{'}}$ be some uniform metric space with $2k^{'}$ points. We now prove the following result regarding the competitive ratio of any randomized online algorithm for the problem with $k^{'}$ clients on $\mathcal{U}_{2k^{'}}$.

Lemma 2.17. *Let $k^{'}$ be a power of 2, i.e. $k^{'} = 2^g$ for some integer $g \geq 0$. The competitive ratio of any randomized online algorithm against an oblivious adversary for the problem with $k^{'}$ clients on $\mathcal{U}_{2k^{'}}$ is at least $\Omega(\log \log k)$.*

Proof. We use Yao's method (see Section 1.1.1.1) for deriving a lower bound on the expected performance of any randomized algorithm on a worst-case input instance by deriving a lower bound on the expected performance of any deterministic algorithm on a particular distribution of random input instances. Let $\mathcal{A}$ be some deterministic online algorithm for the problem with $k^{'}$ clients and let m be an arbitrary positive integer. Let ρ be the uniform distribution over $\mathcal{I}$, the set of all input instances generated by adversary $\mathsf{A}(k^{'}, m)$ from Definition 2.12. Consider the random variable $C^{\mathcal{A}}(\rho)$. For any input instance $I^{'} \in \mathcal{I}$, we divide the cost $C^{\mathcal{A}}(I^{'})$ among the $k^{'}$ clients in the following manner. Whenever the server moves towards some point, the cost of the movement is charged to the client with the longest input sequence who has an outstanding request at that point. Let $C_i(I^{'})$ denote the cost charged to client i. Notice that for any input instance $I^{'} \in \mathcal{I}$, $C_i(I^{'}) = k^{'}m$, if client i is the client with the longest input sequence in $I^{'}$, since no client subsumes this client. Thus, $\mathbb{E}[C_i(\rho)] = k^{'}m$, if client i is the client with the longest input instance. Otherwise, consider a client i with $f = hm$ requests that is subsumed by x clients[3]. By construction, each of the x clients that subsume client i must have at least f requests. Let $B(i, f)$ denote the event that the f-th request of client i is served before the f-th request of any of the x clients that subsume client i. If $B(i, f)$ occurs, the cost of serving client i is $f = hm$. We lower bound the expected cost of serving client i, $\mathbb{E}[C_i(\rho)]$, by f times the probability that $B(i, f)$ occurs. This probability is

[3] By construction, h is a power of 2 and $1 \leq h < k^{'}$.

$\frac{1}{x+1}$ because these x clients and client i are equivalent to the online algorithm until the f-th request of any of them is served. Therefore,

$$\mathbb{E}\left[C_i(\rho)\right] \geq \frac{f}{x+1} = \frac{hm}{x+1}. \tag{2.4.1}$$

For any instance $I^{'} \in \mathcal{I}$, the number of clients with an input sequence of length $f = hm$[4] that are subsumed by exactly x clients equals to the number of nodes[5] at depth x with exactly h descendants in a binomial tree of size $k^{'}$, which is

$$\binom{\log k^{'} - 1 - \log h}{x-1}.$$

Combining (2.4.1) with the fact that for any instance $I^{'} \in \mathcal{I}$ both the number of clients with an input sequence of length $f = hm = 2^j m$ that are subsumed by exactly x clients is $\binom{\log k^{'} - 1 - \log h}{x-1}$ and $C^{\mathcal{A}}(I^{'}) = \sum_{i=1}^{k^{'}} C_i(I^{'})$ holds, we obtain:

$$\begin{aligned}
\mathbb{E}\left[C^{\mathcal{A}}(\rho)\right] &\geq k^{'}m + \sum_{j=0}^{\log k^{'}-1} 2^j m \sum_{x=1}^{\log k^{'}-j} \binom{\log k^{'} - 1 - j}{x-1} \frac{1}{x+1} \\
&= k^{'}m + m \sum_{j=0}^{\log k^{'}-1} 2^j \cdot \frac{\left(\log k^{'} - 1 - j\right) \cdot 2^{\log k^{'} - j} + 1}{(\log k^{'} - j) \cdot (\log k^{'} - j + 1)} \\
&\geq k^{'}m + k^{'}m \sum_{j=0}^{\log k^{'}-1} \frac{\log k^{'} - 1 - j}{(\log k^{'} - j) \cdot (\log k^{'} - j + 1)} \\
&= k^{'}m + k^{'}m \sum_{j=0}^{\log k^{'}-1} \frac{j}{(j+1) \cdot (j+2)} = k^{'}m + k^{'}m \cdot \Theta\left(\log\log k^{'}\right) \\
&= \left(1 + \Theta\left(\log\log k^{'}\right)\right) \cdot k^{'}m.
\end{aligned}$$

On the other hand, for any instance $I^{'} \in \mathcal{I}$, the optimal offline solution has cost $k^{'}m$. This cost is obtained by serving the client with the longest input sequence (which consists of $k^{'}m$ requests), along with the requests of other clients. This solution obtains the optimal cost, the cost of serving the longest client sequence, as the optimal cost cannot be reduced by having more client to serve. In addition, for any instance $I^{'} \in \mathcal{I}$, $C^{\mathcal{OPT}}(I^{'}) = k^{'}m$ can be made arbitrary large by increasing enough the integer m. Hence, from Yao's method and from Lemma 1.11, it follows that the competitive ratio of any randomized online algorithm against an oblivious adversary for the problem with $k^{'}$ clients on $\mathcal{U}_{2k'}$ is at least $1 + \Theta\left(\log\log k^{'}\right) = \Omega\left(\log\log k^{'}\right)$. ■

[4]That is, an input sequence consisting of h different concatenated sequences of length m, where each of them consists of two unique alternating points.

[5]Here, each node represents a possible concatenated sequence of length m, for which there are two unique alternating points.

The desired result follows directly from Lemma 2.17.

Theorem 2.18 (Randomized Lower Bound of $\Omega(\log\log k)$)**.** *Let $k = k' + r$ where $k' = 2^{\lfloor \log k \rfloor}$ and $0 \leq r \leq 2^{\lfloor \log k \rfloor} - 1$. The competitive ratio of any randomized online algorithm against an oblivious adversary for the k-client problem on $\mathcal{U}_{2k'}$ is at least $\Omega(\log\log k)$.*

Proof. Let $\mathcal{A}$ be some randomized online algorithm for the k-client problem on $\mathcal{U}_{2k'}$ and let m be an arbitrary positive integer. Fix an order of the clients from 1 to k and consider the following input instance $I = \langle I_1, \ldots, I_k \rangle$. The sequences of client 1 through client k', i.e. $I_1, \ldots, I_{k'}$, are generated using the construction of adversary $\mathsf{ADV}(k', m)$ (see Definition 2.12). The remaining r sequences are exact copies of I_1. Notice that algorithm $\mathcal{A}$ for the k-client problem on instance I collapses into the k'-client problem on instance $I' = \langle I_1, \ldots, I_{k'} \rangle$. From the proof of Lemma 2.17, we have that the competitive ratio of $\mathcal{A}$ on $\mathcal{U}_{2k'}$ is at least $1 + \Theta\left(\log\log k'\right) = 1 + \Theta(\log\lfloor \log k \rfloor)$. Therefore, the competitive ratio of $\mathcal{A}$ for the k-client problem on $\mathcal{U}_{2k'}$ is at least $1 + \Theta(\log\lfloor \log k \rfloor) = \Omega(\log\log k)$. ■

2.5 General Upper Bounds for Lazy Algorithms

We are now prove that any lazy online algorithm for the uniform k-client problem is already k-competitive w.r.t. two cost functions: the total distance and the average completion time.

Theorem 2.19 (Lazy Algorithms: Total Distance)**.** *Any lazy online algorithm for the k-client problem on any uniform metric space is strictly k-competitive w.r.t. the total distance cost function.*

Proof. Let $\mathcal{A}$ be an online lazy algorithm for the uniform k-client problem. Fix an order of the clients from 1 to k and consider an input instance $I = \langle I_1, \ldots, I_k \rangle$ that consists of requests in some uniform metric space. Let $\widetilde{n}_i$ denote the number of sequences of identical non-dummy requests in the set of requests of client i, $I_i \subseteq I$, for $1 \leq i \leq k$. Notice that the the total distance traveled by the server during the execution of $\mathcal{A}$, i.e. $C^{\mathcal{A}}(I)$, is at most $\sum_{i=1}^{k} \widetilde{n}_i$, by definition of $\widetilde{n}_i$ and since $\mathcal{A}$ is lazy. Indeed, since $\mathcal{A}$ is lazy, in each move towards a point, the server services requests as long as there are requests at that point and services at least one request.

Now, note that for any $1 \leq i \leq k$, $\widetilde{n}_i$ is exactly the optimal cost of serving the set of requests of client i, i.e. $\widetilde{n}_i = C^{\mathcal{OPT}}(I_i)$, since all the requests are in an uniform metric space. We have

$$C^{\mathcal{OPT}}(I) \geq \widetilde{n}_i, \text{ for any } 1 \leq i \leq k, \tag{2.5.1}$$

since $\widetilde{n}_i$ is exactly the optimal cost of serving the set of requests of client i and it cannot be greater than the optimal cost of serving all the requests in I (due

to the triangle inequality). Therefore, we obtain the following upper bound on $C^{\mathcal{A}}(I)$.

$$C^{\mathcal{A}}(I) \leq \sum_{i=1}^{k} \widetilde{n}_i \leq \sum_{i=1}^{k} C^{\mathcal{OPT}}(I) = k \cdot C^{\mathcal{OPT}}(I),$$

where the last inequality follows from (2.5.1). Hence the proof. ■

Theorem 2.20 (Lazy Algorithms: Average Completion Time)**.** *Any lazy online algorithm for the k-client problem on any uniform metric space is strictly k-competitive w.r.t. the average completion time cost function.*

Proof. Let $\mathcal{A}$ be an online lazy algorithm for the uniform k-client problem. Fix an order of the clients from 1 to k and consider an input instance $I = \langle I_1, \ldots, I_k \rangle$ that consists of requests in some uniform metric space. Let $\widetilde{n}_i$ denote the number of sequences of identical non-dummy requests in the set of requests of client i, $I_i \subseteq I$, for $1 \leq i \leq k$. Notice that the number of moves of the server is at most $\sum_{i=1}^{k} \widetilde{n}_i$, by definition of $\widetilde{n}_i$ and since $\mathcal{A}$ is lazy. Indeed, since $\mathcal{A}$ is lazy, in each move towards a point, the server services requests as long as there are requests at that point and services at least one request.

We bound $C^{\mathcal{A}}(I)$ by bounding the completion time of each client of I. For $1 \leq j \leq k$, the completion time of client j is at most $\sum_{i=1}^{k} \widetilde{n}_i$. This bound considers the worst-case scenario. In this scenario, algorithm $\mathcal{A}$ terminates serving the sequence of requests of client j last after $\sum_{i=1}^{k} \widetilde{n}_i$ moves of the server. Therefore, we obtain the following upper bound on $C^{\mathcal{A}}(I)$.

$$C^{\mathcal{A}}(I) \leq \left(\frac{1}{k}\right) \cdot \sum_{j=1}^{k} \sum_{i=1}^{k} \widetilde{n}_i = \left(\frac{1}{k}\right) \cdot k \cdot \sum_{i=1}^{k} \widetilde{n}_i =$$

$$k \cdot \left(\frac{1}{k} \sum_{i=1}^{k} \widetilde{n}_i\right) \leq k \cdot C^{\mathcal{OPT}}(I),$$

where the last inequality follows from Claim 2.6. Hence the proof. ■

Chapter 3

The k-Client Problem With a Serving Taxi

We now introduce a new variation of the basic k-client problem, the *k-client 1-taxi problem*. Let $\mathcal{M} = (M, d)$ be a metric space. In the new variation, there is only one *taxi server* and we are given an *input instance* $I = \langle I_1, \ldots, I_k \rangle$ that consists of k input sequences of requests. For each $1 \leq i \leq k$, the input sequence $I_i = (r_{i,0}, r_{i,1}, \ldots, r_{i,n_i})$ is of length n_i where $r_{i,j} = (s_{i,j}, t_{i,j}) \in M \times M$ and $r_{i,0} = (s_{i,0}, s_{i,0})$ is a dummy request that is associated with the initial position of the serving taxi. An algorithm must move the serving taxi to serve the requests in the following manner. At any time, each client has at most one pending request at the system; more precisely, request $r_{i,j+1}$ arrives exactly when $r_{i,j}$ has been served for $1 \leq i \leq k$ and $0 \leq j \leq n_i - 1$. To serve request $r_{i,j}$, the taxi server moves first to the start $s_{i,j}$ of the request and then to the destination $t_{i,j}$. In the k-client 1-taxi problem, the goal is to minimize the cost of processing all k input sequences, according to the total distance cost function. That is, the cost function is the total distance moved by the serving taxi. In addition, We call requests of the form (s, s) *simple requests* and requests of the form (s, t) with $s \neq t$ *shifted requests*.

Note 3.1. If all the requests are simple, then the k-client 1-taxi problem reduces to the basic k-client problem.

We now show the following connection between the k-client 1-taxi problem and the k-client problem. The result does not depend on the underlying metric space $\mathcal{M}$.

Theorem 3.2 (Connection Between the Taxi Variation and the Basic Problem)**.** *The difference between the best possible deterministic and/or randomized (against oblivious adversaries) competitive ratios of the k-client 1-taxi problem and the basic k-client problem is at most* 2.

Proof. Notice that the k-client 1-taxi problem is a generalization of the basic k-client problem, since the k-client problem restricted to input instances with

only simple requests reduces to the basic k-client problem. Therefore, it is obvious that the competitive ratio of the k-client 1-taxi problem is at least that of the basic k-client problem. Namely, if $\mathcal{A}$ is an α-competitive algorithm for the k-client 1-taxi problem, then $\mathcal{A}$ is also an α-competitive algorithm for the basic k-client problem. Hence, it is sufficient to show that given an α-competitive algorithm $\mathcal{A}$ for the basic k-client problem, it is possible to construct an $(\alpha+2)$-competitive algorithm $\mathcal{A}_T$ for the k-client 1-taxi problem. For simplicity, we prove this only for deterministic algorithm and later explain how to extend this for randomized algorithms (against oblivious adversaries).

The concept of algorithm $\mathcal{A}_T$ is to use a simulation of the actions of $\mathcal{A}$ on the k request sequences. This can be done by replacing each taxi request (s,t) by two successive server requests s and t. So, the conversion of a taxi input instance I_{taxi} on $\mathcal{M}$ to a server input instance I_{server} on $\mathcal{M}$ is done by replacing for each of the k request sequences each taxi request (s,t) with a pair of subsequent server requests s,t , with distance $d(s,t)$ between them. Let $\mathcal{OPT}_T$ be an optimal offline algorithm for the taxi varient and let $\mathcal{OPT}$ be an optimal offline algorithm for the basic k-client problem. We have

$$C^{\mathcal{OPT}}(I_{\text{server}}) \leq C^{\mathcal{OPT}_{\mathcal{T}}}(I_{\text{taxi}}), \tag{3.0.1}$$

due to the optimality of $\mathcal{OPT}$ and since an optimal schedule for I_{taxi} can be converted to a valid schedule for I_{server} of the same cost by setting the server to serve the two associated server requests s,t instead of serving a taxi request (s,t). Thus, since $\mathcal{A}$ is an α-competitive algorithm for the basic k-client problem on $\mathcal{M}$,

$$\begin{aligned} C^{\mathcal{A}}(I_{\text{server}}) &\leq \alpha \cdot C^{\mathcal{OPT}}(I_{\text{server}}) + \tau \\ &\underbrace{\leq}_{(3.0.1)} \alpha \cdot C^{\mathcal{OPT}_{\mathcal{T}}}(I_{\text{taxi}}) + \tau \end{aligned} \tag{3.0.2}$$

for some constant $\tau \geq 0$.

More precisely, the notation of algorithm $\mathcal{A}_T$ is to convert a schedule of $\mathcal{A}$ for I_{server} to a valid schedule for I_{taxi}, while maintaining an additional cost of at most $2 \cdot C^{\mathcal{OPT}_T}(I_{\text{taxi}})$. The $\mathcal{A}_T$ algorithm is detailed in Algorithm 3.1.

Algorithm 3.1 The $\mathcal{A}_T$ Algorithm

function MAIN($p_0, I, \mathcal{A}$)
  Input: initial position of the server p_0
    input instance I that consists of k client sequences starting at p_0
    algorithm $\mathcal{A}$ for the basic k-client problem
  Output: output schedule
  $\mathcal{R} \leftarrow \{\}$ ▷ Initializing the set of clients $\mathcal{R}$
  while there is at least one request to be served **do**
    simulate a move of $\mathcal{A}$ and let $p^{\mathcal{A}}$ be the current server position of $\mathcal{A}$
    add to $\mathcal{R}$ all clients who have an outstanding request with a start at $p^{\mathcal{A}}$
    `ServeByStart`($p^{\mathcal{A}}, \mathcal{R}$) ▷ Serving all requests with a start at $p^{\mathcal{A}}$
    move the taxi to $p^{\mathcal{A}}$
  end while
end function

Function ServeByStart

function SERVEBYSTART($s, \mathcal{R}$)
  Input: start point s
    set of clients $\mathcal{R}$
  while $|\mathcal{R}| \neq 0$ **do**
    for each $i \in \mathcal{R}$ **do**
      let (s_i, t_i) be the outstanding request of client i and move the taxi to s_i and then to t_i
      ▷ Serving the request (s_i, t_i)
      if $|\mathcal{R}| > 1$ **then**
        move the taxi to s
      end if
      $\mathcal{R} \leftarrow \mathcal{R} - \{i\}$ ▷ Updating the unserved clients in $\mathcal{R}$
    end for
  end while
end function

Since for each of the k request sequences of I_{taxi} each taxi request (s, t) is replaced by a pair of server request in the corresponding request sequence of I_{server}, whenever $p^{\mathcal{A}}$ is at the start s of some request (s, t), the serving taxi of $\mathcal{A}_T$ moves to s and then to t. Moreover, during such a move the taxi services only requests of clients from the set of clients served by the server of $\mathcal{A}$. Thus, $\mathcal{A}_T$ indeed returns a valid schedule for I_{taxi}. It remains to analyze the cost of $\mathcal{A}_T$. According to $\mathcal{A}_T$, whenever a request (s, t) has to be served, $p^{\mathcal{A}}$ is at s and the total cost of serving the request and returning the taxi to the location according to the simulation is $d(s, t) + d(t, s) \overbrace{=}^{\text{by symmetry of } \mathcal{M}} 2 \cdot d(s, t)$. Over all k request sequences of I_{taxi}, the total cost of the servings and the returnings is 2 times the sum of the distances of all start-destination pairs in all k request

sequences of I_{taxi}. Since the optimal algorithm $\mathcal{OPT}_{\mathcal{T}}$ must pay at least of all of these distances and since $\mathcal{A}_T$ must pay the cost of the simulation of $\mathcal{A}$ (which is upper bounded by $C^{\mathcal{A}}(I_{\text{server}})$), we have

$$\begin{aligned} C^{\mathcal{A}_T}(I_{\text{taxi}}) &\leq C^{\mathcal{A}}(I_{server}) + 2 \cdot C^{\mathcal{OPT}_{\mathcal{T}}}(I_{\text{taxi}}) \\ &\underbrace{\leq}_{(3.0.2)} (\alpha + 2) \cdot C^{\mathcal{OPT}_{\mathcal{T}}}(I_{\text{taxi}}) + \tau. \end{aligned}$$

Finally, in order to extend the result for randomized algorithms (against oblivious adversaries), we only need to replace $C^{\mathcal{A}_T}$ and $C^{\mathcal{A}}$ by their expectation. ■

Conclusion 3.3. Since there are $(2k-1)$-competitive algorithms for the basic k-client problem on any metric space [1], it can be concluded from the proof of Theorem 3.2 that there exist $(2k+1)$-competitive algorithms for the k-client 1-taxi problem on any metric space.

Chapter 4

The k-Client l-Buffer Problem

We are now define another new variation of the basic k-client problem, the *k-client l-buffer problem*. Let $\mathcal{M} = (M, d)$ be a metric space. In the new variation, there is one *server* and an *input instance* $I = \langle I_1, \ldots, I_k \rangle$ consisting of k input requests is given. For each $1 \leq i \leq k$, the input sequence $I_i = (r_{i,0}, r_{i,1}, \ldots, r_{i,n_i})$ is of length n_i where $r_{i,j} \in M$ and $r_{i,0}$ is a dummy request that is associated with the initial position of the server. In addition, we are given an a *reordering buffer* which is a random access buffer with a storage for l requests that can be used by an online algorithm in order to rearrange the stored requests. Moreover, for each $1 \leq i \leq k$, I_i has at most one *pending unstored* request at the system. That is, for each $1 \leq i \leq k$ and $0 \leq j \leq n_i - 1$, request $r_{i,j+1}$ becomes a pending unstored request of I_i (until it is stored in the buffer) exactly when $r_{i,j}$ has been stored in the buffer. Each pending unstored request of one of the k input sequences, can be stored in the buffer, as long as the buffer is not full. Once the buffer is full, at least one of the requests currently stored in the buffer must be removed and processed by the algorithm. To process a request stored in the buffer, the algorithm moves the server in the metric space to the point corresponding to that request. The removed requests create an *output sequence*. We suppose that the reordering buffer is initially empty, and after processing all k input sequences, the buffer is empty again. In the k-client l-buffer problem, the goal is to minimize the cost of processing all k input sequences, according to the total distance cost function. That is, the cost function is the total distance traveled by the server.

4.1 The General Algorithm

Let us now show a connection between the k-client l-buffer problem and RBM with a buffer of size l. More precisely, we describe how to combine any deterministic algorithm for RBM with a buffer of size l with the k-client $\mathcal{SEQ}$ algorithm

to derive a deterministic competitive k-client l-buffer algorithm. Given any deterministic competitive algorithm $\mathcal{A}$ for RBM with a buffer of size l combined with the k-client $\mathcal{SEQ}$ algorithm, the obtained deterministic algorithm $\mathcal{A}$-$\mathcal{SEQ}$ defined as follows.

Definition 4.1 (The $\mathcal{A}$-$\mathcal{SEQ}$ Algorithm)**.** The $\mathcal{A}$-$\mathcal{SEQ}$ selects an order of the clients from 1 to k and uses the reordering buffer of size l for servicing the current client using the deterministic algorithm $\mathcal{A}$ for RBM with a buffer of size l. After all the requests of the current client are processed, the server moves back to its initial position unless it is the last client.

We can obtain the following result on the competitive ratio of $\mathcal{A}$-$\mathcal{SEQ}$. Note that if $\mathcal{A}$ is a competitive algorithm on some metric space, then $\mathcal{A}$-$\mathcal{SEQ}$ is also a competitive algorithm on this metric space. The $\mathcal{A}$-$\mathcal{SEQ}$ algorithm is described in Algorithm 4.1.

Algorithm 4.1 The $\mathcal{A}$-$\mathcal{SEQ}$ Algorithm

Require: initial position of the server p_0
  k clients who each have a sequence of requests starting at p_0
  deterministic algorithm $\mathcal{A}$ for RBM with a buffer of size l
select an order of the clients from 1 to k.
for $i \leftarrow 1$ to $k-1$ **do**
  serve all the requests of the i-th client using $\mathcal{A}$
  move the server back to its initial position p_0
end for
serve all the requests of the k-th client using $\mathcal{A}$

Theorem 4.2 (Competitive Ratio of $\mathcal{A}$-$\mathcal{SEQ}$)**.** *If $\mathcal{A}$ is a deterministic α-competitive algorithm for RBM with a buffer of size l, then $\mathcal{A}$-$\mathcal{SEQ}$ is a deterministic $(2k-1)\alpha$-competitive algorithm for the k-client l-buffer problem.*

Proof. Given an input instance I, let the clients be ordered according to the order chosen by the $\mathcal{A}$-$\mathcal{SEQ}$ algorithm. Recall that $I_i \subseteq I$ denotes the set of requests of client i for $1 \leq i \leq k$. We have

$$C^{\mathcal{A}\text{-}\mathcal{SEQ}}(I) \leq C^{\mathcal{A}}(I_k) + \sum_{i=1}^{k-1} 2C^{\mathcal{A}}(I_i) = \sum_{i=1}^{k} C^{\mathcal{A}}(I_i) + \sum_{i=1}^{k-1} C^{\mathcal{A}}(I_i). \tag{4.1.1}$$

This inequality holds since the cost for $\mathcal{A}$-$\mathcal{SEQ}$ to return the server to its initial position after processing all requests of client i for $1 \leq i \leq k-1$ is upper bounded by $C^{\mathcal{A}}(I_i)$. Also, since $\mathcal{A}$ is α-competitive for RBM with a buffer of size l and by definition of I_i, there exists a non-negative constant τ that is independent of I_i such that $C^{\mathcal{A}}(I_i) \leq \alpha \cdot C^{\mathcal{OPT}}(I_i) + \tau$ for $1 \leq i \leq k$. In addition, it is obvious that $C^{\mathcal{OPT}}(I_i) \leq C^{\mathcal{OPT}}(I)$ for $1 \leq i \leq k$, as the optimal cost cannot be reduced by having more client to serve. Combining the last two inequalities, we have

$$C^{\mathcal{A}}(I_i) \leq \alpha \cdot C^{\mathcal{OPT}}(I) + \tau, \text{ for } 1 \leq i \leq k. \tag{4.1.2}$$

Plugging (4.1.2) back into (4.1.1) gives:

$$C^{\mathcal{A}\text{-}\mathcal{SEQ}}(I) \leq (2k-1)\alpha \cdot C^{\mathcal{OPT}}(I) + (2k-1)\tau.$$

Setting $\widetilde{\tau} = (2k-1)\tau$, we have

$$C^{\mathcal{A}\text{-}\mathcal{SEQ}}(I) \leq (2k-1)\alpha \cdot C^{\mathcal{OPT}}(I) + \widetilde{\tau}.$$

Note that $\widetilde{\tau}$ is a non-negative constant that is independent of the input instance I, since τ is independent of I_i for all $1 \leq i \leq k$. Hence the proof. ■

Note 4.3. In the case where $\mathcal{M}$ is bounded[1], $\mathcal{A}$-$\mathcal{SEQ}$ is in fact $k\alpha$-competitive for the k-client l-buffer problem. Since $\mathcal{M}$ is bounded, suppose that the distance between any two points in $\mathcal{M}$ is at most D. In order to show that $\mathcal{A}$-$\mathcal{SEQ}$ is $k\alpha$-competitive, the only change that has to be made is using the following inequality instead of inequality (4.1.1):

$$C^{\mathcal{A}\text{-}\mathcal{SEQ}}(I) \leq C^{\mathcal{A}}(I_k) + \sum_{i=1}^{k-1} \left(C^{\mathcal{A}}(I_i) + D\right) = \sum_{i=1}^{k} C^{\mathcal{A}}(I_i) + (k-1)\,D. \quad (4.1.3)$$

This inequality holds since the cost for $\mathcal{A}$-$\mathcal{SEQ}$ to return the server to its initial position after processing all requests of client i for $1 \leq i \leq k-1$ is upper bounded by D. Plugging (4.1.2) into (4.1.3) gives:

$$C^{\mathcal{A}\text{-}\mathcal{SEQ}}(I) \leq k\alpha \cdot C^{\mathcal{OPT}}(I) + k\tau + (k-1)\,D.$$

Setting $\widetilde{\tau} = k\tau + (k-1)\,D$, we have

$$C^{\mathcal{A}\text{-}\mathcal{SEQ}}(I) \leq k\alpha \cdot C^{\mathcal{OPT}}(I) + \widetilde{\tau}.$$

Note that $\widetilde{\tau}$ is a non-negative constant that is independent of the input instance I, since τ is independent of I_i for all $1 \leq i \leq k$. Thus, $\mathcal{A}$-$\mathcal{SEQ}$ is $k\alpha$-competitive in the case where $\mathcal{M}$ is bounded.

4.1.1 Implications

With Theorem 4.2 and the results mentioned in Section 1.3.2 for RBM with a buffer of size l, we obtain the following results for various algorithms and metric spaces.

Corollary 4.4. *On any uniform metric space, the $\mathcal{A}$-$\mathcal{SEQ}$ algorithm is $O\left(k\sqrt{\log l}\right)$-competitive, where $\mathcal{A}$ is the algorithm presented in [12].*

Corollary 4.5. *On a line metric space with n evenly spaced points, the $\mathcal{A}$-$\mathcal{SEQ}$ algorithm is $O\left(k \log n\right)$-competitive, where $\mathcal{A}$ is the algorithm presented in [9].*

[1] That is, there exists a real number $D > 0$ such that for any two points $p, q \in M$, $d(p, q) \leq D$.

Corollary 4.6. *On any bounded metric space, the $\mathcal{A}$-$\mathcal{SEQ}$ is $k\,(2l-1)$-competitive, where $\mathcal{A}$ is is an algorithm that processes the requests stored in the buffer according to the order of their arrival (that is, $\mathcal{A}$ does not use the buffer to reorder the stored requests).*

Corollary 4.7. *On any bounded metric space, the $\mathcal{A}$-$\mathcal{SEQ}$ is $(2k-1)\,(2l-1)$-competitive, where $\mathcal{A}$ is is an algorithm that processes the requests stored in the buffer according to the order of their arrival (that is, $\mathcal{A}$ does not use the buffer to reorder the stored requests).*

Corollary 4.8. *On an n-point space with aspect ratio Δ, the $\mathcal{A}$-$\mathcal{SEQ}$ algorithm is $O\left(k\log\Delta + k\cdot\min\left\{\log l, \log n\right\}\right)$-competitive, where $\mathcal{A}$ is the algorithm presented in [23].*

Chapter 5

Discussion

In this work, we have studied the *uniform k-client problem*, which is a special case of the k-client problem where the underlying metric space is uniform. Some of our work here is based on [1]. For the total distance cost function, we have extended the deterministic and randomized lower bound results from [1] so that they are adapted to the usual definition of competitive ratio. More precisely, we derived a deterministic lower bound of $\frac{\lfloor \log k \rfloor}{2} + 1 = \Omega(\log k)$ and substantially improved the previously known bound of $2 - \frac{1}{k}$, which was proven in [2]. As for the randomized case, we obtained a lower bound of $\Omega(\log \log k)$ and a constant lower bound of 1.25 (which holds even for 2 clients). There is still a gap between the k upper bound and the $\Omega(\log k)$ lower bound for the total distance and average completion time cost functions, which leaves a great opening for further research. Is there an online algorithm better than k-competitive, or is there a lower bound of k? For the total distance cost function, the picture is unclear even for randomized algorithms, as the best known upper bound is still k whereas there is a lower bound of $\Omega(\log \log k)$.

In this work, we have only deduced which online algorithms cannot surely have a competitive ratio better than k. These online algorithms are $\mathcal{SEQ}$, $\mathcal{CPF}$ and $\mathcal{RSEQ}$ and they are standard algorithms with quite simple analysis. A possible direction of study may be examining other online algorithms (both deterministic and randomized) with more advanced analysis, in order to develop an online algorithm better than k-competitive. For example, one can investigate using *potential function* arguments the performance of the Random Choice ($\mathcal{RC}$) algorithm. This is a randomized online algorithm, which at any given time randomly selects a request from among all the outstanding requests and processes it.

Considering the theoretical results obtained so far for the uniform k-client problem with the total distance cost function, it may be interesting to evaluate the performance of several algorithms (including the algorithms mentioned in the previous paragraph) on random input instances and examine theory against practice. This is done before with other related problems (e.g., in [25], for the

reordering buffer management problem).

Furthermore, we have mentioned the MTP problem, a generalized version of the uniform k-client problem where there multiple servers. In [2], the problem was studied only for the total distance cost function and there is a huge gap between the lower and upper bounds for deterministic online algorithms. It seems possible to improve the lower bound, as we did in the case of a single server. However, designing an online algorithm with a competitive ratio better than the trivial upper bound is not a walk in the park, even in the case of a single server.

In this work, we have also considered two variations of the k-client problem. One of them is the *k-client 1-taxi problem*, a generalized version of the k-client problem with the total distance cost function, where there is a serving taxi and each request is associated with a pair of points in the metric space (start point and destination point). Essentially, we have shown that this variation is not significantly more complex than the basic k-client problem. A possible research direction here may be exploring the k-client 1-taxi problem with other cost functions (e.g., the average completion time, or the total distance moved by the serving taxi without considering its movements from a start point to the corresponding destination point).

Bibliography

[1] H. Alborzi, E. Torng, P. Uthaisombut, and S. Wagner. The k-client problem. In *J. Algorithms*, 41(2):115-173, 2001.

[2] E. Feuerstein and A. Strejilevich de Loma. On-line multi-threaded paging. In *Algorithmica*, 32(1):36-60, 2002.

[3] A.C.-C. Yao. Probabilistic computations: Toward a unified measure of complexity. In *Proceedings of the 18th IEEE Symposium on Foundations of Computer Science*, pp. 222-227, 1977.

[4] H. Räcke, C. Sohler, and M. Westermann. Online scheduling for sorting buffers. In *Proceedings of the 10th European Symposium on Algorithms (ESA)*, pp. 820-832, 2002.

[5] I. Gamzu and D. Segev. Improved online algorithms for the sorting buffer problem. In *Proceedings of the 24th Symposium on Theoretical Aspects of Computer Science (STACS)*, pp. 658-669, 2007.

[6] H.-L. Chan , N. Megow, R. Stee, and R. Sitters. The sorting buffer problem is NP-hard. CoRR, abs/1009.4355, 2010.

[7] Y. Asahiro, K. Kawahara, and E. Miyano. NP-hardness of the sorting buffer problem on the uniform metric. In *Discrete Applied Mathematics*, pp. 1453-1464, 2010.

[8] S.S. Seiden. Randomized online multi-threaded paging. In *Nordic J. Comput.*, 6(2):148-161, 1999.

[9] M. Englert. The reordering buffer problem on the line revisited. In *ACM SIGACT News*, 49(1):67-72, 2018.

[10] M. Englert and M. Westermann. Reordering buffers management for non-uniform cost models. In *Proc. ICALP'05*, pp. 627-638, 2005.

[11] N. Avigdor-Elgrabli and Y. Rabani. An improved competitive algorithm for reordering buffer management. In *Proc. SODA 2010*, pp. 13-21, 2010.

[12] A. Adamaszek, A. Czumaj, M. Englert, and H. Räcke. Almost tight bounds for reordering buffer management. In *Proc. STOC 2011*, pp. 607-616, 2011.

[13] N. Avigdor-Elgrabli and Y. Rabani. An optimal randomized online algorithm for reordering buffer management. In *Proceedings of the 54th IEEE Symposium on Foundations of Computer Science (FOCS)*, pp. 1-10, 2013.

[14] N. Avigdor-Elgrabli and Y. Rabani. A constant factor approximation algorithm for reordering buffer management. In *Proceedings of the 24th ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 973-984, 2013.

[15] M. Englert and M. Westermann. Reordering buffer for non-uniform cost models. In *Proceedings of the 32nd International Colloquium on Automata, Languages and Programming (ICALP)*, pp. 556-564, 2007.

[16] R. Khandekar and V. Pandit. Online and offline algorithms for the sorting buffers problem on the line metric. In *Journal of Discrete Algorithms*, 8(1):24-35, 2010.

[17] M. Englert, H. Räcke, and M. Westermann. Reordering buffers for general metric spaces. In *Theory of Computing*, 6(1):27-46, 2010.

[18] M. Englert and H. Räcke. Reordering Buffers with Logarithmic Diameter Dependency for Trees. In *Proceedings of the 28th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 1224-1234, 2017.

[19] J. Fakcharoenphol, S. Rao, and K. Talwar. A tight bound on approximating arbitrary metrics by tree metrics. In *J. Com put. System Sci.,* 69(3):485-497, 2004.

[20] D.D. Sleator and R.E. Tarjan. Amortized efficiency of list update and paging rules. In *Communications of ACM*, 28:202-208, 1985.

[21] A. Borodin and R. El-Yaniv. Online Computation and Competitive Analysis. *Cambridge University Press*, 1998.

[22] S. Ben-David, A. Borodin, R. Karp, G. Tardos, and A. Wigderson. On the power of randomization in online algorithms. In *Algorithmica*, 11:73-91, 1994.

[23] M. Kohler and H. Räcke. Reordering Buffer Management with a Logarithmic Guarantee in General Metric Spaces. In *44th International Colloquium on Automata, Languages, and Programming (ICALP 2017)*, 33:1-12, 2017.

[24] V. Rehn-Sonigo, D. Trystram, F. Wagner, H. Xu, and G. Zhang. Offline scheduling of multi-threaded request streams on a caching server. In *International Parallel and Distributed Processing Symposium*, pp. 1167-1176, 2011.

[25] M. Englert, H. Röglin, and M. Westermann. Evaluation of online strategies for reordering buffers. In *ACM Journal of Experimental Algorithmics*, 14, pp. 3.3–3.14, 2009.